



UNIVERSIDAD
DE GRANADA

Facultad de Ciencias

GRADO EN MATEMÁTICAS

TRABAJO DE FIN DE GRADO

Fundamentos y aplicaciones del descenso por gradiente en aprendizaje automático

Presentado por:
Lidia Nievas Dueñas

Curso académico 2025-2026



Fundamentos y aplicaciones del descenso por gradiente en aprendizaje automático

Lidia Nievas Dueñas

Lidia Nievas Dueñas *Fundamentos y aplicaciones del descenso por gradiente en aprendizaje automático.*

Trabajo de fin de Grado. Curso académico 2025-2026.

**Responsable de
tutorización**

Pablo Morales Álvarez
*Departamento de Estadística e Investigación
Operativa*

Grado en Matemáticas
Facultad de Ciencias
Universidad de Granada

DECLARACIÓN DE ORIGINALIDAD

D./Dña. Lidia Nievas Dueñas

Declaro explícitamente que el trabajo presentado como Trabajo de Fin de Grado (TFG), correspondiente al curso académico 2025-2026, es original, entendido esto en el sentido de que no he utilizado para la elaboración del trabajo fuentes sin citarlas debidamente.

En Granada a 12 de junio de 2026

Fdo: Lidia Nievas Dueñas

A mi familia y amigos, por apoyarme en cada etapa de mi vida.

Agradecimientos

Me gustaría empezar estas líneas echando la vista atrás y recordando a todas y cada una de las personas -familiares, amigos y personal docente- que me han acompañado durante todos estos años de formación, aprendizaje y crecimiento personal.

En primer lugar, no puedo evitar pensar en mi madre, mi apoyo incondicional durante toda mi vida. Mi primera mejor amiga, mi mayor referente y la persona que siempre ha estado ahí para mí, acompañándome en cada etapa, igual que sé que seguirá haciendo en todas las que queden por venir. Mamá, te agradezco infinitamente todo lo que has hecho por mí. Gracias por la mujer en la que me has convertido. Todo esto es, en gran parte, gracias a ti.

Siguiendo este orden, también quiero agradecer al resto de mi familia. Tengo la suerte de contar con una familia a la que admiro y admiraré siempre. A mi hermano, por quien lo daría todo con tal de verle feliz; a Vicen, culpable en gran parte de que hoy esté estudiando esto; a mis abuelos y a mi tía, que forman una parte esencial de mí; y a mi padre.

Por otro lado, si hoy estoy aquí, escribiendo los agradecimientos de mi Trabajo Fin de Grado del Grado en Matemáticas, es sin duda gracias a todos los profesores que, a lo largo de mi recorrido académico, me han transmitido el amor por las matemáticas y han despertado en mí esa curiosidad que me ha traído hasta aquí. Don Manuel, Carmen y Luisa, gracias a vosotros hoy estoy a un paso de convertirme en matemática.

Quiero agradecer también a mi tutor, Pablo, por su ayuda y orientación durante la realización de este trabajo. Gracias por acompañarme en este proceso.

No puedo olvidarme de todas las personas que me han acompañado durante este tiempo y que, aunque no son familia, durante estos cinco años de carrera se han convertido en algo muy parecido.

Desde el primer día de clase tuve la suerte de formar un grupo de amigas como el que nunca antes había tenido. Juntas hemos compartido mil anécdotas, fiestas, clases y un millón de horas de estudio. Gracias, Alba, Mayte, Pili, Lucía, Marga y Mari; sin vosotras todo esto habría sido muy diferente.

Un tiempo después se unieron todos los demás, y juntos formamos un grupo que ha pasado por tantas y tantas asignaturas, comidas en el comedor, casas rurales, barriles e innumerables experiencias compartidas.

En tercero de carrera tomé una de las mejores decisiones de mi vida: irme de Erasmus. Aquella experiencia no solo me abrió la mente y me hizo aprender muchísimo, tanto a nivel académico como personal, sino que también me regaló a personas que hoy son imprescindibles en mi vida. Gracias a todos por hacer de esos nueve meses los mejores de mi vida. Desde entonces me sé de memoria el camino a una casa que ya no es la mía. Siempre habrá un trocito de todos nosotros en Nancy.

Por último, no podía cerrar estos agradecimientos sin acordarme de todas esas personas que, aunque hoy nuestros caminos hayan tomado direcciones distintas, han sido una pieza

Agradecimientos

fundamental durante este tiempo. Y también de aquellas que se están incorporando recientemente y que, de una forma u otra, empiezan a formar parte de mí.

Hoy, al cerrar esta etapa, me siento muy orgullosa de quien soy y de todo lo vivido. Han sido muchos días buenos y otros no tan buenos; muchos momentos de agobio, dudas e incertidumbre, pero de una forma u otra, siempre he conseguido salir adelante, en gran parte gracias a la compañía que tenía al lado. Durante estos años he aprendido matemáticas, pero también he aprendido a confiar, a resistir, a crecer y a entender que cada persona y cada experiencia dejan algo en nosotros.

Dejo esta universidad siendo matemática, pero lo más importante que me llevo es el corazón lleno.

A todos y cada uno de vosotros, gracias. Sin vosotros no sería quien soy hoy, y todo lo que consiga es tan mío como vuestro.

Summary

Mathematical optimization is a cornerstone of modern machine learning. Most learning models, from classical regressions to deep neural networks, are trained by minimizing a loss function that evaluates model parameters over a given dataset. For a dataset $\{(x_i, y_i)\}_{i=1}^n$ and parameters $\theta \in \mathbb{R}^d$, this training process can be formulated as the large-scale minimization problem:

$$f(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta).$$

This dissertation provides a comprehensive theoretical and practical study of gradient-based optimization methods, focusing on how the geometric properties of the objective function determine algorithmic efficiency, convergence rates, and solution quality.

The first part establishes the mathematical framework by analyzing convex optimization and the structural properties of functions. Properties such as convexity, smoothness, and strong convexity are introduced and differentiated through gradients and Hessian matrices. In convex settings, every local minimum is a global minimum, making rigorous convergence analysis tractable.

We analyze classical gradient descent (GD) under varying geometric assumptions. In the general convex case, GD exhibits a sublinear rate, requiring $\mathcal{O}(1/\varepsilon^2)$ iterations to achieve an ε -error. If smoothness is assumed, the bound improves to $\mathcal{O}(1/\varepsilon)$. Furthermore, when the objective is both smooth and strongly convex, GD achieves linear convergence, reducing the iteration complexity to $\mathcal{O}(\log(1/\varepsilon))$. We also study Nesterov's accelerated gradient descent, which achieves the optimal first-order rate of $\mathcal{O}(1/\sqrt{\varepsilon})$ via momentum strategies and potential functions, as well as subgradient methods for non-differentiable convex optimization.

To address the computational challenges of massive datasets where computing the exact gradient is infeasible, we analyze stochastic gradient descent (SGD) and mini-batch strategies. We prove that under conditional expectation, the stochastic gradient remains an unbiased estimator of the true gradient. While SGD inherits deterministic convergence properties in expectation, the inherent sampling noise creates a variance threshold (*noise floor*) near the optimum, justifying the practical use of decreasing learning rates and mini-batches. Additionally, we explore non-convex optimization, showing that while global guarantees disappear due to saddle points and local minima, GD can still achieve a $\mathcal{O}(1/T)$ convergence to a stationary point, highlighting the critical role of initialization and trajectory dynamics.

The second part of the dissertation validates these theoretical bounds empirically. We implement and evaluate these first-order algorithms on classic machine learning problems using real-world datasets: linear regression (Diabetes dataset), L_1 -regularized logistic regression (Breast Cancer Wisconsin dataset), and a multilayer perceptron neural network (California Housing dataset).

The experimental results are consistent with the theoretical predictions. In convex settings, empirical error curves match the proved sublinear, linear, and accelerated convergence slopes. In stochastic regimes, the experiments illustrate the computational trade-off between

Summary

step size and gradient variance. Finally, neural network training confirms that the gradient norms decrease according to non-convex analytical trends despite the stochastic noise.

In conclusion, this work demonstrates that the efficiency of gradient-based optimization is deeply intertwined with the geometry of the loss surface. While convexity, smoothness, and strong convexity dictate fundamental efficiency bounds, stochasticity and non-convexity introduce challenges that require sophisticated analytical and empirical tuning, cementing first-order methods at the core of modern machine learning success.

Índice general

Agradecimientos	v
Summary	vii
Introducción	xi
I. Fundamentos teóricos	1
1. Fundamentos matemáticos de la optimización	3
1.1. Descripción de un problema de optimización	3
1.2. Optimización convexa	4
1.3. Caracterización diferencial de la convexidad	6
1.4. Minimización de funciones convexas: existencia y unicidad	8
2. Descenso por gradiente en funciones convexas	11
2.1. Descenso por gradiente tradicional (Batch)	12
2.1.1. Deducción de la dirección de descenso	12
2.1.2. Análisis del error acumulado	12
2.1.3. Análisis de complejidad y tasas de convergencia	13
2.1.4. Convergencia en escenarios de curvatura acotada	14
2.1.5. Convergencia en escenarios de suavidad y convexidad fuerte	15
2.1.6. Aceleración: El método de Nesterov	20
2.2. Descenso por subgradiente	22
2.2.1. El concepto de subdiferenciabilidad	22
2.2.2. Dinámica del algoritmo	24
2.3. Descenso por gradiente estocástico	26
2.3.1. Inssegadez del gradiente estocástico	27
2.3.2. Análisis de convergencia estocástica	28
2.3.3. Generalización al caso no diferenciable	30
2.3.4. Estrategia de <i>mini-batches</i>	31
2.4. Optimalidad de los métodos de primer orden	32
3. Descenso por gradiente en funciones no convexas	33
3.1. Funciones suaves no convexas	33
3.2. Análisis de la trayectoria	34
3.2.1. Redes neuronales lineales profundas	34
3.2.2. Una función no convexa modelo	35
3.2.3. Suavidad a lo largo de la trayectoria	36
3.2.4. Convergencia	36
3.2.5. Dependencia de la inicialización	37
3.3. Análisis final	37

4. Conclusiones teóricas	39
 II. Experimentación práctica	 41
5. Experimentación en el caso convexo	43
5.1. Validación empírica de las tasas de convergencia	43
5.1.1. Caso convexo Lipschitz: $\mathcal{O}(1/\sqrt{T})$	43
5.1.2. Caso suave convexo: tasa $\mathcal{O}(1/T)$	44
5.1.3. Caso suave y fuertemente convexo: tasa $\mathcal{O}((1 - \frac{\mu}{L})^T)$	45
5.1.4. Aceleración de Nesterov: tasa $\mathcal{O}(1/T^2)$	46
5.1.5. Descenso de subgradiente sobre funciones no diferenciables	47
5.1.6. Descenso estocástico y <i>mini-batches</i>	47
5.2. Aplicaciones al aprendizaje automático	48
5.2.1. Regresión lineal: caso convexo	48
5.2.2. Regresión logística con regularización L_1 : caso no diferenciable	49
 6. Experimentación no convexa	 51
6.1. Validación empírica de las cotas no convexas	51
6.1.1. Función suave no convexa: tasa $\mathcal{O}(1/T)$	51
6.1.2. Análisis de la trayectoria: tasa exponencial sobre la red lineal profunda	52
6.2. Aplicación a redes neuronales	53
6.2.1. Breve introducción a las redes neuronales	53
6.2.2. Implementación para una red neuronal multicapa	54
 7. Conclusiones de la experimentación	 55
 Bibliografía	 57

Introducción

La optimización matemática constituye uno de los pilares principales del aprendizaje automático moderno [3, 2, 1]. La gran mayoría de modelos utilizados actualmente, desde métodos clásicos de regresión hasta redes neuronales profundas, pueden interpretarse como problemas de minimización de una función de pérdida dependiente de un conjunto de parámetros.

Dado un modelo con parámetros $\theta \in \mathbb{R}^d$ y un conjunto de datos de entrenamiento $\{(x_i, y_i)\}_{i=1}^n$, esta función toma la forma

$$f(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta),$$

donde cada f_i mide la discrepancia entre la predicción del modelo y el dato real (x_i, y_i) .

Entrenar un modelo no es más que resolver este problema de optimización, por lo que con este trabajo se busca responder a la siguiente pregunta: ¿cómo se resuelve eficientemente, y qué garantías matemáticas podemos dar sobre la velocidad y la calidad de la solución encontrada?

La herramienta usada para contestar a esta pregunta es el **descenso por gradiente**, cuya idea fundamental se remonta a Cauchy [4]: si en cada iteración nos desplazamos desde el punto actual en la dirección opuesta al gradiente de f , la función decrece localmente de forma óptima. A pesar de su antigüedad, el análisis riguroso de la convergencia de dicha idea no se pudo consolidar hasta los años ochenta, cuando Nesterov [9] estableció tanto las tasas de convergencia alcanzables bajo distintas hipótesis sobre f , como los límites teóricos que ningún método de primer orden puede superar.

A partir de los años 2000, el volumen de datos con el que se trabaja ha crecido exponencialmente, lo que ha obligado a replantear el problema: calcular el gradiente exacto de f en cada iteración puede llegar a ser inviable cuando n es del orden de millones. El **descenso por gradiente estocástico** (SGD), cuyas raíces fueron establecidas por Robbins y Monro [13], resurgió como respuesta a dicho problema y se instauró como herramienta central en el aprendizaje automático moderno.

El objetivo principal de este trabajo ha sido desarrollar un estudio matemático y práctico de los métodos de descenso por gradiente y de sus principales variantes, analizando cómo las propiedades geométricas de la función objetivo condicionan el comportamiento de los algoritmos.

En particular, los objetivos inicialmente planteados fueron:

1. Estudiar el problema de optimización en funciones convexas.
2. Analizar el método de descenso por gradiente clásico.
3. Estudiar el descenso por gradiente estocástico.
4. Aplicar de forma práctica los métodos estudiados.

A lo largo del desarrollo del trabajo, estos objetivos se han ampliado para incluir otros resultados como el descenso por subgradiente o el estudio de la optimización no convexa. Esto ha permitido construir una visión más completa de los algoritmos utilizados actualmente en aprendizaje automático.

El trabajo se ha estructurado en dos grandes partes complementarias. La primera parte desarrolla el marco teórico necesario para comprender el funcionamiento de los métodos de optimización. Se comienza en el Capítulo 1 introduciendo fundamentos matemáticos de la optimización convexa, herramientas que permiten garantizar la existencia y unicidad de soluciones óptimas y facilitan el análisis posterior de los algoritmos iterativos. El resultado central que convierte el problema de optimización en uno tratable es la noción de que una función convexa garantiza que todo mínimo local es un mínimo global.

Tras esto, en el Capítulo 2 se estudia el método de descenso por gradiente clásico, obteniendo distintas tasas de convergencia según las distintas hipótesis impuestas sobre la función objetivo: sin ninguna hipótesis de curvatura, se requieren $O(1/\varepsilon^2)$ iteraciones para alcanzar un error de ε ; bajo suavidad, esta cifra baja a $O(1/\varepsilon)$; y si además la función es fuertemente convexa, la convergencia pasa a ser exponencial, $O(\log(1/\varepsilon))$. El capítulo incluye además el método acelerado de Nesterov, que logra mejorar la tasa del caso suave hasta $O(1/\sqrt{\varepsilon})$ y cuyo teorema de optimalidad demuestra que esta tasa es la mejor posible en métodos de primer orden, el descenso por subgradiente para funciones no diferenciables, y el descenso por gradiente estocástico, donde se demuestra que el SGD hereda las tasas del caso determinista en esperanza siempre que el tamaño de paso se gestione adecuadamente.

Finalmente, el Capítulo 3 aborda el caso no convexo. Aunque en este escenario desaparecen muchas de las garantías globales, se estudian resultados que permiten entender por qué el descenso por gradiente sigue siendo válido a pesar de la no convexidad. En funciones suaves no convexas, solo podemos decir que la norma del gradiente tiende a cero con tasa de $O(1/T)$, por lo que no podemos garantizar la convergencia al mínimo global. Cuando la función posee además una estructura adicional, como el c -balanceo, es posible recuperar tasas de convergencia exponenciales mediante el análisis de trayectoria.

La segunda parte del trabajo tiene un carácter experimental y busca validar empíricamente los resultados teóricos obtenidos. Se implementan los algoritmos sobre funciones modelo y se aplican a problemas concretos de aprendizaje automático: regresión lineal por mínimos cuadrados sobre el conjunto de datos *Diabetes*, regresión logística con el conjunto de datos *Breast Cancer Wisconsin* y una red neuronal perceptrón multicapa sobre el conjunto de datos *California Housing*. En los experimentos sobre una función objetivo convexa se verifica que las curvas de error empírico reproducen las pendientes predichas por la teoría en escala logarítmica y en el caso no convexo se confirma que la norma del gradiente decrece con la tendencia predicha a pesar del ruido introducido por el muestreo aleatorio de *mini-batches*.

Para la realización de este trabajo han sido especialmente relevantes los conocimientos adquiridos por los cursos de Análisis Matemático, Probabilidad y Estadística impartidos durante el grado, siendo especialmente importante la asignatura de *Optimisation – algorithmique* que tuve el placer de cursar durante mi estancia en la Université de Lorraine, que fue la que me hizo descubrir el mundo del aprendizaje automático y me despertó el interés que me ha llevado a realizar este trabajo.

Las principales referencias bibliográficas utilizadas incluyen textos de optimización matemática, siendo [6] el principal recurso utilizado durante la parte teórica y [17] para la parte práctica.

Parte I.

Fundamentos teóricos

1. Fundamentos matemáticos de la optimización

1.1. Descripción de un problema de optimización

Un problema de optimización matemática, o simplemente **problema de optimización** [3], tiene la forma:

$$\text{minimizar } f_0(x) \text{ sujeto a } f_i(x) \leq b_i, \quad i = 1, \dots, m. \quad (1.1)$$

Aquí, el vector $x = (x_1, \dots, x_d)$ es la **variable de optimización** del problema, la función $f_0 : \mathbb{R}^d \rightarrow \mathbb{R}$ es la **función objetivo**, las funciones $f_i : \mathbb{R}^d \rightarrow \mathbb{R}, i = 1, \dots, m$, son las funciones de **restricción** (de desigualdad), y las constantes $b_1, \dots, b_m$ son los **límites** o cotas de dichas restricciones.

Un vector x^* se denomina **óptimo**, o una solución del problema (1.1), si tiene el valor objetivo más pequeño entre todos los vectores que satisfacen las restricciones: para cualquier z con $f_1(z) \leq b_1, \dots, f_m(z) \leq b_m$, se cumple que

$$f_0(z) \geq f_0(x^*).$$

Generalmente consideramos familias o clases de problemas de optimización, caracterizadas por formas particulares de la función objetivo y de las restricciones.

En el contexto del **Aprendizaje Automático** (*Machine Learning*), esta formulación matemática adquiere una interpretación directa: la función objetivo f_0 representa la **función de pérdida** (Loss Function), la cual mide la discrepancia entre las predicciones del modelo y los datos reales. Por su parte, el vector de variables x no es una incógnita abstracta, sino que constituye el conjunto de parámetros o **pesos del modelo** (denotados habitualmente como θ o w) que debemos ajustar para minimizar dicho error. Así, resolver el problema de optimización equivale a «entrenar» al modelo para que alcance su máximo rendimiento predictivo.

Como ejemplo importante, el problema de optimización (1.1) se llama **programa lineal** si la función objetivo y las funciones de restricción $f_0, \dots, f_m$ son afines, es decir, satisfacen:

$$f_i(\alpha x + \beta y) = \alpha f_i(x) + \beta f_i(y) \quad (1.2)$$

para todo $x, y \in \mathbb{R}^d$ y todo $\alpha, \beta \in \mathbb{R}$ tales que $\alpha + \beta = 1$. Si el problema de optimización no es lineal, se llama **programa no lineal**.

En el siguiente capítulo trataremos una clase de problemas de optimización llamados **problemas de optimización convexa**.

Un problema de optimización convexa es aquel en el que la función objetivo y las funciones de restricción son convexas, lo que significa que satisfacen la desigualdad:

$$f_i(\alpha x + \beta y) \leq \alpha f_i(x) + \beta f_i(y) \quad (1.3)$$

1. Fundamentos matemáticos de la optimización

para todo $x, y \in \mathbb{R}^d$ y todo $\alpha, \beta \in \mathbb{R}$ con $\alpha + \beta = 1, \alpha \geq 0, \beta \geq 0$.

Comparando (1.3) y (1.2), vemos que la convexidad es más general que la linealidad: una desigualdad reemplaza a la igualdad más restrictiva, y la desigualdad debe cumplirse solo para ciertos valores de α y β . Dado que cualquier programa lineal es, por lo tanto, un problema de optimización convexa, podemos considerar la optimización convexa como una generalización de la programación lineal.

1.2. Optimización convexa

Definición 1.1. Sea $X \subseteq \mathbb{R}^d$. El conjunto X es **convexo** si y solo si

$$\forall (x, y) \in X, \quad \forall \lambda \in [0, 1], \quad \lambda x + (1 - \lambda)y \in X,$$

es decir, si x y y son dos elementos de X , entonces el segmento que une a x y a y está incluido en X , tal y como se ve en la Figura 1.1.

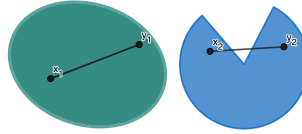


Figura 1.1.: Conjunto convexo (izquierda) y conjunto no convexo (derecha)
(imagen de creación propia usando GeoGebra).

En el contexto del aprendizaje automático, asegurar que el espacio X sea convexo garantiza que los segmentos entre puntos factibles siguen siendo factibles y facilita que los mínimos locales sean globales

Ahora, veamos esta misma definición aplicada a funciones:

Definición 1.2. Sea $f: X \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$ una función, diremos que es **convexa** si:

(i) Su dominio X es convexo.

(ii) $\forall (x, y) \in X, \quad \forall \lambda \in [0, 1], \quad f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$.

Geométricamente, esta definición significa que el segmento que conecta $(x, f(x))$ y $(y, f(y))$ se encuentra puntualmente por encima del gráfico de f (Figura 1.2).

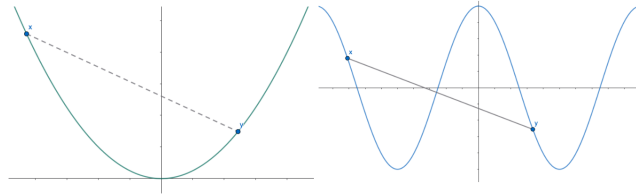


Figura 1.2.: Función convexa (izquierda) y función no convexa (derecha)
(imagen de creación propia usando GeoGebra).

Antes de pasar a la siguiente definición, veamos un lema interesante que nos será clave para próximos resultados [16]:

Lema 1.3 (Desigualdad de Jensen). Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa, $x_1, x_2, \dots, x_m \in \text{dom}(f)$ y $\lambda_1, \dots, \lambda_m \in \mathbb{R}_+$ tales que $\sum_{i=1}^m \lambda_i = 1$. Entonces, $f\left(\sum_{i=1}^m \lambda_i x_i\right) \leq \sum_{i=1}^m \lambda_i f(x_i)$.

Demostración. Procederemos por inducción sobre el número de puntos m .

Paso base ($m = 2$): Por la definición de función convexa, para cualquier $\lambda_1, \lambda_2 \geq 0$ con $\lambda_1 + \lambda_2 = 1$:

$$f(\lambda_1 x_1 + \lambda_2 x_2) \leq \lambda_1 f(x_1) + \lambda_2 f(x_2)$$

Hipótesis inductiva: Supongamos que la desigualdad se cumple para $m - 1$ puntos.

Paso inductivo: Consideremos el caso para m puntos. Sea $\sum_{i=1}^m \lambda_i x_i$ una combinación convexa. Si $\lambda_m = 1$, el resultado es trivial. Si $\lambda_m < 1$, podemos reescribir la suma de la siguiente manera:

$$\sum_{i=1}^m \lambda_i x_i = (1 - \lambda_m) \left(\sum_{i=1}^{m-1} \frac{\lambda_i}{1 - \lambda_m} x_i \right) + \lambda_m x_m.$$

Definimos $\tilde{\lambda}_i = \frac{\lambda_i}{1 - \lambda_m}$. Notemos que $\sum_{i=1}^{m-1} \tilde{\lambda}_i = \frac{1}{1 - \lambda_m} \sum_{i=1}^{m-1} \lambda_i = \frac{1 - \lambda_m}{1 - \lambda_m} = 1$.

Ahora, aplicando la propiedad de convexidad para el caso base ($m = 2$):

$$f\left((1 - \lambda_m) \sum_{i=1}^{m-1} \tilde{\lambda}_i x_i + \lambda_m x_m\right) \leq (1 - \lambda_m) f\left(\sum_{i=1}^{m-1} \tilde{\lambda}_i x_i\right) + \lambda_m f(x_m)$$

Como la suma dentro de f en el primer término del lado derecho tiene $m - 1$ elementos que cumplen las condiciones de combinación convexa, aplicamos la **hipótesis inductiva**:

$$f\left(\sum_{i=1}^{m-1} \tilde{\lambda}_i x_i\right) \leq \sum_{i=1}^{m-1} \tilde{\lambda}_i f(x_i)$$

Sustituyendo esto en nuestra desigualdad:

$$\begin{aligned} f\left(\sum_{i=1}^m \lambda_i x_i\right) &\leq (1 - \lambda_m) \left(\sum_{i=1}^{m-1} \frac{\lambda_i}{1 - \lambda_m} f(x_i) \right) + \lambda_m f(x_m) \\ &= \sum_{i=1}^{m-1} \lambda_i f(x_i) + \lambda_m f(x_m) \\ &= \sum_{i=1}^m \lambda_i f(x_i) \end{aligned}$$

Esto completa la demostración. □

Aparte de la definición de convexidad que hemos visto anteriormente, podemos encontrar una aún más fuerte:

1. Fundamentos matemáticos de la optimización

Definición 1.4. Una función $f: X \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$ se dice **estrictamente convexa** si:

(i) X es convexo.

(ii) $\forall x \neq y \in X \quad \forall \lambda \in]0, 1[\quad , f(\lambda x + (1 - \lambda)y) < \lambda f(x) + (1 - \lambda)f(y)$

Esto equivale a decir que el segmento que une a $(x, f(x))$ con $(y, f(y))$ está estrictamente por encima del gráfico de f .

Lema 1.5. Supongamos que $\text{dom}(f)$ es abierto y que f es dos veces continuamente diferenciable. Si el Hessiano es definido positivo ($\nabla^2 f(x) \succ 0 \forall x \in \text{dom}(f)$), entonces f es estrictamente convexa.

Observación 1.6. El recíproco es falso, por ejemplo $f(x) = x^4$ es estrictamente convexa pero su segunda derivada se anula en $x = 0$.

Más allá del formalismo de estas definiciones, en el contexto del aprendizaje automático, la convexidad y la curvatura (dada por la Hessiana) garantizan que el gradiente apunte claramente hacia el único punto óptimo, ya que la función de pérdida tendrá «forma de valle», lo que garantiza que cualquier mínimo local sea el óptimo global. Esto evita que el modelo «dude» entre diferentes soluciones que parecen igual de buenas, proporcionando una dirección de descenso clara y única hacia la solución.

Además, una curvatura mínima (convexidad fuerte) garantiza que las paredes de dicho «valle» sean empinadas, lo que empuja al gradiente a converger mucho más rápido como veremos en los próximos capítulos.

1.3. Caracterización diferencial de la convexidad

En el caso de que la función f sea diferenciable (de primer o segundo orden) en todo punto, decimos que podemos caracterizar la convexidad de la función por su gradiente o por su Hessiana.

Esto es muy útil ya que normalmente es más fácil trabajar con estas caracterizaciones que con la propia definición. Estas caracterizaciones pueden consultarse en [3].

Lema 1.7 (Caracterización de primer orden de la convexidad). Supongamos que $\text{dom}(f)$ es abierto y que f es diferenciable. En particular, el gradiente

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x), \frac{\partial f}{\partial x_2}(x), \dots, \frac{\partial f}{\partial x_d}(x) \right)$$

existe para todo punto $x \in X$. Entonces f es convexa si y solo si $\text{dom}(f)$ es convexo y

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) \quad \forall x, y \in \text{dom}(f)$$

Demostración. Supongamos que f es convexa, esto es, para $t \in]0, 1[$,

$$f(x + t(y - x)) = f((1 - t)x + ty) \leq (1 - t)f(x) + tf(y) = f(x) + t(f(y) - f(x)).$$

Dividiendo por t y utilizando la diferenciabilidad en x , obtenemos:

$$\begin{aligned} f(y) &\geq f(x) + \frac{f(x + t(y - x)) - f(x)}{t} = f(x) + \frac{\nabla f(x)^\top t(y - x) + r(t(y - x))}{t} \\ &= f(x) + \nabla f(x)^\top (y - x) + \frac{r(t(y - x))}{t}, \end{aligned}$$

donde el término de error $r(t(y - x))/t \rightarrow 0$ cuando $t \rightarrow 0$. De aquí se sigue la desigualdad:

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x).$$

Ahora supongamos que esta desigualdad se cumple para todos los $x, y \in \text{dom}(f)$. Sean $\lambda \in [0, 1]$ y definamos $z := \lambda x + (1 - \lambda)y \in \text{dom}(f)$ (por la convexidad de $\text{dom}(f)$). Entonces tenemos:

$$f(x) \geq f(z) + \nabla f(z)^\top (x - z), \quad (1.4)$$

$$f(y) \geq f(z) + \nabla f(z)^\top (y - z). \quad (1.5)$$

Tras multiplicar la primera desigualdad (1.4) por λ y la segunda (1.5) por $(1 - \lambda)$, los términos del gradiente se cancelan al sumar ambas desigualdades, y obtenemos:

$$\lambda f(x) + (1 - \lambda)f(y) \geq f(z) = f(\lambda x + (1 - \lambda)y).$$

Esto demuestra la convexidad. $\square$

Este resultado es fundamental para entender por qué el descenso por gradiente funciona.

Nos asegura que el gradiente local nos da información global. Si seguimos la dirección opuesta al gradiente, estamos garantizando que nos movemos hacia valores menores de la función de pérdida.

Esta caracterización se puede transformar de forma sencilla en lo que comúnmente se conoce como *Monotonía del gradiente*:

Lema 1.8 (Monotonía del gradiente). *Supongamos que $\text{dom}(f)$ es abierto y que f es diferenciable. Entonces f es convexa si y solo si $\text{dom}(f)$ es convexo y*

$$(\nabla f(y) - \nabla f(x))^\top (y - x) \geq 0 \quad \forall x, y \in \text{dom}(f).$$

Si además $f : \text{dom}(f) \rightarrow \mathbb{R}$ es dos veces diferenciable (lo que significa que f es diferenciable y la función gradiente ∇f también es diferenciable), se da lo que conocemos como la segunda caracterización de la convexidad:

Lema 1.9 (Caracterización de segundo orden de la convexidad). *Supongamos que $\text{dom}(f)$ es abierto y que f es dos veces diferenciable; en particular, la matriz Hessiana (matriz de segundas derivadas parciales)*

$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1}(x) & \frac{\partial^2 f}{\partial x_1 \partial x_2}(x) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_d}(x) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(x) & \frac{\partial^2 f}{\partial x_2 \partial x_2}(x) & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_d}(x) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_d \partial x_1}(x) & \frac{\partial^2 f}{\partial x_d \partial x_2}(x) & \cdots & \frac{\partial^2 f}{\partial x_d \partial x_d}(x) \end{pmatrix}$$

1. Fundamentos matemáticos de la optimización

existe en cada punto $x \in \text{dom}(f)$ y es simétrica. Entonces, f es convexa si y solo si $\text{dom}(f)$ es convexo y, para todo $x \in \text{dom}(f)$, tenemos que $\nabla^2 f(x) \succeq 0$ (es decir, $\nabla^2 f(x)$ es semidefinida positiva).

Demostración. Usamos la misma función h definida anteriormente, $h(t) := f(x + t(y - x))$, para $x, y \in \text{dom}(f)$ fijos y $t \in I$, donde $I \supseteq [0, 1]$ es un intervalo abierto adecuado. Pero esta vez, también necesitamos la segunda derivada de h . Para $t \in I$ y $v := y - x$, tenemos:

$$\begin{aligned} h'(t) &= \nabla f(x + tv)^T v \\ h''(t) &= v^T \nabla^2 f(x + tv) v. \end{aligned}$$

Si f es convexa, siempre tenemos $h''(0) \geq 0$, como mostraremos a continuación. Dado esto, $\nabla^2 f(x) \succeq 0$ se deduce para cada $x \in \text{dom}(f)$: debido a que $\text{dom}(f)$ es abierto, para cada $v \in \mathbb{R}^d$ de norma suficientemente pequeña, existe un $y \in \text{dom}(f)$ tal que $v = y - x$, y entonces $v^T \nabla^2 f(x) v = h''(0) \geq 0$. Por escalado, esta desigualdad se extiende a todo $v \in \mathbb{R}^d$.

Para mostrar que $h''(0) \geq 0$, observamos que para todo ε suficientemente pequeño, $x + \varepsilon v \in \text{dom}(f)$ y, por lo tanto:

$$\frac{h'(\varepsilon) - h'(0)}{\varepsilon} = \frac{(\nabla f(x + \varepsilon v) - \nabla f(x))^T v}{\varepsilon} \geq 0$$

por la monotonía del gradiente para funciones convexas. Se sigue que $h''(0) = \lim_{\varepsilon \rightarrow 0} (h'(\varepsilon) - h'(0))/\varepsilon \geq 0$.

Para la otra dirección, el teorema del valor medio aplicado a h' produce un $c \in (0, 1)$ tal que $h'(1) - h'(0) = h''(c)$, y desarrollado, esto es:

$$\nabla f(y)^T v - \nabla f(x)^T v = v^T \nabla^2 f(x + cv) v \geq 0,$$

dado que $\nabla^2 f(z) \succeq 0$ para todo $z \in \text{dom}(f)$. Por lo tanto, hemos demostrado la monotonía del gradiente, lo que implica la convexidad de f . $\square$

Geométricamente, esto significa que la función tiene una curvatura no negativa en todas las direcciones, similar a la forma de un cuenco. En algunos modelos, demostrar que la Hessiana de la función de coste es semidefinida positiva es lo que nos permite asegurar que el entrenamiento convergerá al óptimo global.

1.4. Minimización de funciones convexas: existencia y unicidad

La convexidad es la noción que nos va a dar información sobre el carácter global de un punto mínimo.

Teorema 1.10 (Condición suficiente de optimización global). Sea $X \subseteq \mathbb{R}^d$ un conjunto convexo y $f: X \rightarrow \mathbb{R}$ una función. Sea x^* un punto de mínimo local de f .

1. Si f es convexa, entonces x^* es un punto de mínimo global de f , es decir,

$$f(x^*) \leq f(y) \quad \forall y \in X.$$

2. Si f es estrictamente convexa, entonces x^* es el único punto de mínimo global de f en X .

Demostración. 1. Lo haremos por reducción al absurdo. Supongamos que existe $y \in X$ tal que $f(y) < f(x^*)$ y sea $y' = \lambda x^* + (1 - \lambda)y$ para $\lambda \in [0, 1]$.

Por convexidad tenemos que $f(y') < f(x^*)$.

Tomando λ muy cerca de 1 tal que $\|y' - x^*\| < \varepsilon$, llegamos a una contradicción con que x^* sea mínimo local.

2. Supongamos que $x^* \neq y^*$ son dos mínimos globales con $f_{\min} = f(x^*) = f(y^*)$, y sea $z = \frac{1}{2}x^* + \frac{1}{2}y^*$. Por ser f estrictamente convexa, $f(z) < \frac{1}{2}f_{\min} + \frac{1}{2}f_{\min} = f_{\min}$, lo que contradice que x^* y y^* sean mínimos globales. $\square$

Este es el resultado más potente de la optimización convexa para el aprendizaje automático. Nos garantiza que, bajo las hipótesis anteriores, si nuestro algoritmo de descenso encuentra un punto donde la pendiente es cero hemos alcanzado el error mínimo absoluto.

Además, en el contexto del *machine learning*, esto nos asegura que con encontrar un punto óptimo local, el error del modelo es el mínimo absoluto.

En conclusión, obtenemos el siguiente resultado:

Corolario 1.11. Sea $X \subset \mathbb{R}^d$ un conjunto no vacío, compacto y convexo, y f continua y estrictamente convexa; entonces f alcanza un único mínimo global. Alternativamente, si X es cerrado y f es coerciva, también hay existencia. [3]

Observación 1.12. Esto no significa que una función convexa siempre tenga un mínimo global. Lo podemos ver simplemente en la función $f(x) = x$.

Para asegurar la existencia de mínimo global, tenemos que añadir condiciones adicionales. Algunas de ellas las podemos ver en los siguientes resultados.

Lema 1.13. Supongamos que $f: X \rightarrow \mathbb{R}$ es convexa y diferenciable sobre $X \subseteq \mathbb{R}^d$ abierto. Sea $x \in X$. Si $\nabla f(x) = 0$, entonces x es un mínimo global.

Demostración. Supongamos que $\nabla f(x) = 0$. Según el Lema 1.7 tenemos

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) = f(x) \quad \forall y \in X,$$

es decir, x es un mínimo global. $\square$

El inverso también es cierto y no requiere convexidad.

Lema 1.14. Supongamos que $f: X \rightarrow \mathbb{R}$ es diferenciable sobre $X \subseteq \mathbb{R}^d$ abierto. Sea $x \in X$. Si x es un mínimo global, entonces $\nabla f(x) = 0$.

Demostración. Supongamos que $\nabla f(x)_i \neq 0$ para algún i . Para $t \in \mathbb{R}$, definimos $x(t) = x + te_i$, donde e_i es la i -ésima componente del vector unitario. Para un $|t|$ suficientemente pequeño, tenemos $x(t) \in X$ ya que X es abierto. Sea $z(t) = f(x(t))$. Por la regla de la cadena,

$$z'(0) = \nabla f(x)^\top e_i = \nabla f(x)_i \neq 0$$

Por tanto, z decrece alejándose de 0, lo que implica $f(x(t)) < f(x)$ para algún t , por lo que x no es un mínimo global. $\square$

1. Fundamentos matemáticos de la optimización

Los resultados anteriores establecen que, para una función de pérdida convexa y diferenciable, la condición $\nabla f(x^*) = 0$ es necesaria y suficiente para garantizar un óptimo global. En la práctica, esto transforma el problema de «entrenar» un modelo en el problema de encontrar las raíces del gradiente.

Sin embargo, como bien apunta la observación sobre la función $f(x) = x$, la convexidad por sí sola no garantiza que dicho mínimo exista o sea alcanzable en un tiempo finito. En el entrenamiento de modelos complejos, esto nos obligará a introducir conceptos como la suavidad (para controlar grandes saltos del gradiente) y la convexidad fuerte (para asegurar que el «valle» sea lo suficientemente pronunciado), los cuales definiremos al inicio del próximo capítulo para analizar la velocidad de nuestros algoritmos.

2. Descenso por gradiente en funciones convexas

El algoritmo de **descenso por gradiente** es el más útil en el contexto de minimizar funciones. Es extremadamente simple y sorprendentemente robusto en el sentido de que también funciona bien para funciones que no son convexas.

Sin embargo, la gran mayoría de resultados teóricos solo son válidos para funciones convexas.

Para simplificar la notación de aquí en adelante, haremos algunas suposiciones:

Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa y diferenciable. Asumimos que x^* es un mínimo global de f . Nuestro objetivo va a ser buscar una aproximación de x^* . Esto normalmente significa que para un cierto $\varepsilon > 0$, buscamos

$$x \in \mathbb{R}^d \quad \text{tal que} \quad f(x) - f(x^*) < \varepsilon.$$

El método de **descenso por gradiente** (GD) es un método iterativo, lo que quiere decir que genera una secuencia de soluciones en cada iteración hasta que se cumpla

$$f(x_T) - f(x^*) < \varepsilon.$$

Observación 2.1. A lo largo de este capítulo, la tasa de convergencia tras T iteraciones y la complejidad para alcanzar una precisión ε se relacionan mediante la equivalencia

$$f(x_T) - f(x^*) \leq C/T \implies T \geq C/\varepsilon.$$

Así, una tasa $\mathcal{O}(1/T)$ equivale a una complejidad $\mathcal{O}(1/\varepsilon)$; una tasa acelerada $\mathcal{O}(1/T^2)$ equivale a $\mathcal{O}(1/\sqrt{\varepsilon})$; y una tasa exponencial $(1 - \mu/L)^T$ equivale a $\mathcal{O}(\log(1/\varepsilon))$.

La siguiente tabla nos da una primera vista de lo que vamos a ver. Contiene varias variantes del método de descenso por gradiente y el número de pasos que se requiere en la clase de funciones correspondiente para lograr un error de aproximación aditivo de a lo sumo ε .

	Convexas Lipschitz	Convexas Suaves	Fuertemente Convexas	Suaves y Fuert. Convexas
Descenso por gradiente	Thm. 2.2 $\mathcal{O}(1/\varepsilon^2)$	Thm. 2.6 $\mathcal{O}(1/\varepsilon)$		Thm. 2.10 $\mathcal{O}(\log(1/\varepsilon))$
Gradiente acelerado		Thm. 2.11 $\mathcal{O}(1/\sqrt{\varepsilon})$		
Descenso por subgradiente	Thm. 2.17 $\mathcal{O}(1/\varepsilon^2)$		Thm. 2.19 $\mathcal{O}(1/\varepsilon)$	
Gradiente estocástico	Thm. 2.20 $\mathcal{O}(1/\varepsilon^2)$		Thm. 2.22 $\mathcal{O}(1/\varepsilon)$	

Tabla 2.1.: Resultados en las distintas variaciones del descenso por gradiente

2. Descenso por gradiente en funciones convexas

En este capítulo, analizaremos algunas de sus versiones y cómo las propiedades geométricas de la función dictan la velocidad del método.

La principal fuente consultada para la elaboración de este capítulo es [6].

2.1. Descenso por gradiente tradicional (Batch)

El algoritmo de **descenso por gradiente** es un algoritmo muy simple para buscar la aproximación deseada de x^* bajo ciertas condiciones. Da como resultado una serie $x_0, x_1, \dots$ de vectores tales que x_0 es arbitrario y, para cada $t \geq 0$, x_{t+1} es obtenido a partir de x_t por la siguiente expresión:

$$x_{t+1} = x_t + v_t \quad \text{para un cierto } v_t \in \mathbb{R}^d.$$

2.1.1. Deducción de la dirección de descenso

Para garantizar que el entrenamiento progrese buscamos que $f(x_{t+1}) < f(x_t)$. Utilizando la **diferenciabilidad de f** y considerando un desplazamiento v_t pequeño, la aproximación de Taylor de primer orden nos indica:

$$f(x_t + v_t) = f(x_t) + \nabla f(x_t)^\top v_t + r(v_t) \approx f(x_t) + \nabla f(x_t)^\top v_t.$$

Para maximizar el descenso en cada iteración bajo una norma de paso constante, debemos elegir v_t de modo que el producto escalar $\nabla f(x_t)^\top v_t$ sea lo más negativo posible. Por la desigualdad de Cauchy-Schwarz, esto ocurre cuando v_t apunta en la dirección del gradiente negativo.

Así, cada paso del descenso por gradiente está definido por

$$x_{t+1} = x_t - \gamma \nabla f(x_t). \quad (2.1)$$

donde $\gamma > 0$ es un **tamaño de paso** (en la literatura de *machine learning*, se le suele llamar *learning rate*) determinado, pero también tendría sentido que dependiera de t , como veremos más tarde.

Por ahora lo dejamos fijado, aunque su elección será muy importante ya que, si es muy pequeño, el proceso puede hacerse muy largo y si es demasiado grande corremos riesgo de que diverja.

A lo largo de este capítulo veremos cómo adaptarlo bien en cada situación.

2.1.2. Análisis del error acumulado

Denotemos ε_t al **error** en la iteración t , en el contexto de minimización.

Solemos considerar $\varepsilon_t = f(x_t) - f(x^*)$, pero puede haber otras medidas de error.

Para cuantificar el rendimiento del algoritmo, analizamos el error respecto al valor óptimo $f(x^*)$. Utilizando la caracterización de primer orden de la convexidad (lema 1.7), sabemos que:

$$f(x_t) - f(x^*) \leq \nabla f(x_t)^\top (x_t - x^*). \quad (2.2)$$

Así, hemos reducido el problema a calcular $\nabla f(x_t)^\top (x_t - x^*)$, y eso será lo siguiente que hagamos:

Sea x_t alguna iteración de la secuencia (2.1). Llamemos $g_t = \nabla f(x_t)$. Por definición de descenso por gradiente,

$$g_t = \frac{x_t - x_{t+1}}{\gamma} \Rightarrow g_t^\top (x_t - x^*) = \frac{1}{\gamma} (x_t - x_{t+1})^\top (x_t - x^*).$$

Ahora, usando la identidad $\|a - b\|^2 = \|a\|^2 - 2a^\top b + \|b\|^2$, podemos reescribir la expresión anterior como

$$\begin{aligned} g_t^\top (x_t - x^*) &= \frac{1}{2\gamma} (\|x_t - x_{t+1}\|^2 + \|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2) \\ &= \frac{1}{2\gamma} (\gamma^2 \|g_t\|^2 + \|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2) \\ &= \frac{\gamma}{2} \|g_t\|^2 + \frac{1}{2\gamma} (\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2). \end{aligned}$$

Al sumar esta expresión sobre todas las iteraciones de t , los dos últimos términos dentro del paréntesis se anulan en una suma telescópica:

$$\begin{aligned} \sum_{t=0}^{T-1} g_t^\top (x_t - x^*) &= \frac{\gamma}{2} \sum_{t=0}^{T-1} \|g_t\|^2 + \frac{1}{2\gamma} (\|x_0 - x^*\|^2 - \|x_T - x^*\|^2) \\ &\leq \frac{\gamma}{2} \sum_{t=0}^{T-1} \|g_t\|^2 + \frac{1}{2\gamma} \|x_0 - x^*\|^2. \end{aligned}$$

Haciendo uso de (2.2) obtenemos:

$$\sum_{t=0}^{T-1} (f(x_t) - f(x^*)) \leq \frac{\gamma}{2} \sum_{t=0}^{T-1} \|g_t\|^2 + \frac{1}{2\gamma} \|x_0 - x^*\|^2. \quad (2.3)$$

Esto nos proporciona un límite superior para el error promedio $f(x_t) - f(x^*)$ para $t = 0, \dots, T-1$ y, por tanto, para el error producido por la iteración con el valor de la función más pequeño.

2.1.3. Análisis de complejidad y tasas de convergencia

Esta cota revela una dependencia directa de la distancia inicial al óptimo ($\|x_0 - x^*\|^2$), lo cual es intuitivo. Sin embargo, la presencia del sumatorio de las normas de los gradientes ($\sum \|g_t\|^2$) plantea un problema: si no controlamos la magnitud del gradiente, no podemos asegurar la convergencia del error promedio a cero.

En el aprendizaje automático, esto justifica la necesidad de utilizar funciones con gradientes acotados o Lipschitz-continuos, y explica por qué la última iteración x_T no siempre es la mejor solución, prefiriéndose a veces el promedio de las iteraciones o el punto con el menor valor de pérdida registrado durante el proceso.

Para comparar la eficiencia de los algoritmos de optimización, clasificamos la velocidad con la que el error ε_t decae hacia cero:

- **Lineal:** Un algoritmo se dice que tiene (al menos) **convergencia lineal** cuando existe un número $0 < c < 1$ tal que $\varepsilon_{t+1} \leq c\varepsilon_t$ para un t suficientemente grande. En *machine*

2. Descenso por gradiente en funciones convexas

learning implica una convergencia exponencialmente rápida, característica de funciones fuertemente convexas.

- **Superlineal:** El término **convergencia superlineal** hace referencia a un algoritmo para el cual hay constantes $r > 1$ y $c > 0$ tales que $\varepsilon_{t+1} \leq c(\varepsilon_t)^r$ para un t suficientemente grande.
- **Sublineal:** Si la convergencia es más lenta que la lineal (por ejemplo de orden $\mathcal{O}(1/T)$ o $\mathcal{O}(1/T^2)$). Este es el comportamiento estándar del descenso por gradiente en funciones convexas generales que no presentan una curvatura mínima pronunciada.

2.1.4. Convergencia en escenarios de curvatura acotada

Esta es la solución más fácil para extraer algo de información del análisis anterior. Al no tener ninguna información sobre cómo varía el gradiente, lo mínimo que podemos hacer es pedir que el gradiente no sea infinito (esté acotado).

Supongamos por tanto que todos los gradientes de f (una función diferenciable) tienen una norma acotada, es decir, supongamos que todas las funciones son **Lipschitz-continuas** sobre $\mathbb{R}^d$.

Esto nos descarta muchas funciones interesantes. Por ejemplo, la función convexa modelo $f(x) = x^2$ queda descartada, ya que $\nabla f(x) = 2x$ no está acotada. Sin embargo, veremos que esta hipótesis es muy útil.

Teorema 2.2. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa y diferenciable con un mínimo global x^* . Supongamos que $\|x_0 - x^*\| \leq R$ y $\|\nabla f(x)\| \leq B$ para todo x . Tomando el paso de gradiente de descenso (learning rate) $\gamma = \frac{R}{B\sqrt{T}}$, se tiene

$$\frac{1}{T} \sum_{t=0}^{T-1} (f(x_t) - f(x^*)) \leq \frac{RB}{\sqrt{T}}.$$

Demostración. La prueba resulta bastante simple partiendo de (2.3). Acotando $\|x_0 - x^*\| \leq R$ y $\|g_t\| \leq B$, obtenemos

$$\sum_{t=0}^{T-1} (f(x_t) - f(x^*)) \leq \frac{\gamma}{2} B^2 T + \frac{1}{2\gamma} R^2, \quad (2.4)$$

así que elegimos γ tal que

$$q(\gamma) = \frac{\gamma}{2} B^2 T + \frac{R^2}{2\gamma}$$

es mínima.

Así,

$$q'(\gamma) = \frac{1}{2} B^2 T - \frac{R^2}{2\gamma^2},$$

igualando a 0 tenemos

$$\frac{1}{2} B^2 T = \frac{R^2}{2\gamma^2} \Rightarrow \gamma^2 B^2 T = R^2 \Rightarrow \gamma = \frac{R}{B\sqrt{T}}.$$

Y sustituyendo en (2.4) se tiene el resultado buscado. □

Esto quiere decir que para alcanzar $\min_{t=0}^{T-1} (f(x_t) - f(x^*)) \leq \varepsilon$, necesitamos $T \geq \frac{R^2 B^2}{\varepsilon^2}$ iteraciones, de donde se deduce el número de pasos tomando R y B como constantes.

Es posible que en casos concretos tengamos problemas con esta elección de γ , sobre todo porque no depende de d (la dimensión del espacio). Es cierto que podemos tomar R y B dependientes de d , pero normalmente la dependencia es leve.

Sin embargo, esta independencia respecto a la dimensión d es una de las mayores virtudes del descenso por gradiente en el ámbito del aprendizaje automático de alta dimensión. Mientras que otros métodos de optimización colapsan ante el crecimiento del número de parámetros, el descenso por gradiente mantiene una tasa de convergencia teórica constante, lo que lo hace idóneo para entrenar modelos con millones de pesos.

Desde un punto de vista práctico, una convergencia sublineal no es del todo buena, ya que es bastante lenta. Si queremos el error (por ejemplo) 10 veces más pequeño, el número de iteraciones necesarias aumenta por un factor de 100.

Sin embargo, la robustez de este resultado ante problemas de gran dimensión garantiza que el algoritmo va a converger, independientemente de la complejidad del modelo, siempre que el gradiente se mantenga acotado.

2.1.5. Convergencia en escenarios de suavidad y convexidad fuerte

Para mejorar la tasa de convergencia obtenida en la sección anterior debemos imponer condiciones sobre la curvatura de la función de pérdida.

En el ámbito del aprendizaje automático, esto se traduce en analizar cómo de rápido varía el gradiente. Si el gradiente cambia de forma lenta y controlada, el algoritmo puede dar pasos más grandes y seguros.

Si además la función tiene una curvatura mínima garantizada (convexidad fuerte), el «valle» de la función de pérdida tendrá una forma pronunciada que conducirá al algoritmo hacia el punto óptimo con una velocidad mayor.

2.1.5.1. Funciones suaves

Recordemos que nuestra herramienta principal para el análisis estándar fue la caracterización de primer orden de la convexidad (lema 1.7).

De nuevo, queremos analizar aquellas funciones para las cuales $f(y)$ puede ser acotada superiormente por $f(x) + \nabla f(x)^\top (y - x)$, con un error máximo cuadrático.

La siguiente definición se aplica para todas las funciones diferenciables, no se requiere convexidad:

Definición 2.3. Sea $f: \text{dom}(f) \rightarrow \mathbb{R}$ una función diferenciable, $X \subseteq \text{dom}(f)$ convexo y $L \in \mathbb{R}_+$. La función f es llamada **suave** (con parámetro L) **sobre** X si

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|x - y\|^2 \quad \forall x, y \in X.$$

Si $X = \text{dom}(f)$, f es simplemente llamada suave.

Esto quiere decir que para cualquier $x \in X$, la gráfica de f se encuentra por debajo de un paraboloide tangencial no demasiado pronunciado en $(x, f(x))$ (Figura 2.1).

2. Descenso por gradiente en funciones convexas

Tenemos la siguiente caracterización para funciones suaves:

Lema 2.4. Supongamos que $f : \text{dom}(f) \rightarrow \mathbb{R}$ es una función diferenciable tal que $\text{dom}(f)$ es abierto y convexo. Sea $L \in \mathbb{R}_+$, equivalen:

1. f es suave con parámetro L .
2. g definida por $g(x) = \frac{L}{2}x^\top x - f(x)$ es convexa sobre $\text{dom}(g) = \text{dom}(f)$.

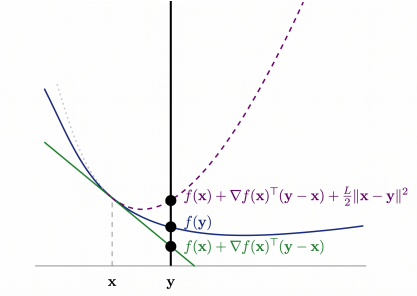


Figura 2.1.: Función suave convexa (imagen generada con Gemini).

Con esta nueva definición, en esta sección ya incluimos funciones que en la sección anterior quedaban excluidas, como $f(x) = x^2$, que es suave con parámetro $L = 2$,

$$f(y) = y^2 = x^2 + 2x(y - x) + (x - y)^2 = f(x) + f'(x)(y - x) + \frac{L}{2}(x - y)^2.$$

A continuación demostraremos que, para funciones convexas suaves, el análisis estándar proporciona una mejor cota que la ya obtenida bajo la hipótesis de gradientes acotados.

Comencemos con un lema previo que demuestra que el descenso por gradiente (con un tamaño de paso γ adecuado) logra progresos en el valor de la función en cada paso sobre funciones suaves.

Denominamos a esto **descenso suficiente** (*sufficient decrease*).

Lema 2.5 (Descenso suficiente). Sea $f : \mathbb{R}^d \rightarrow \mathbb{R}$ diferenciable y suave con parámetro L . Con $\gamma = \frac{1}{L}$, el descenso por gradiente satisface

$$f(x_{t+1}) \leq f(x_t) - \frac{1}{2L} \|\nabla f(x_t)\|^2, \quad t \geq 0.$$

Más concretamente, esto ya se cumple si f es suave con parámetro L sobre el segmento de recta que conecta x_t y x_{t+1} .

Demostración. Aplicamos la condición de suavidad y la definición de descenso por gradiente, de la que sigue

$$x_{t+1} - x_t = -\frac{\nabla f(x_t)}{L}.$$

Tenemos entonces:

$$\begin{aligned}
 f(x_{t+1}) &\leq f(x_t) + \nabla f(x_t)^\top (x_{t+1} - x_t) + \frac{L}{2} \|x_t - x_{t+1}\|^2 \\
 &= f(x_t) - \frac{1}{L} \|\nabla f(x_t)\|^2 + \frac{1}{2L} \|\nabla f(x_t)\|^2 \\
 &= f(x_t) - \frac{1}{2L} \|\nabla f(x_t)\|^2.
 \end{aligned}$$

□

Teorema 2.6. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ función convexa y diferenciable con un mínimo global x^* . Supongamos además que f es suave con parámetro L . Eligiendo un tamaño de paso $\gamma = \frac{1}{L}$, el descenso por gradiente resulta:

$$f(x_T) - f(x^*) \leq \frac{L}{2T} \|x_0 - x^*\|^2, \quad T > 0.$$

Demostración. Aplicamos el descenso suficiente (lema 2.5) para acotar la suma de $\|g_t\|^2 = \|\nabla f(x_t)\|^2$ en el paso (2.3) del análisis estándar:

$$\frac{1}{2L} \sum_{t=0}^{T-1} \|\nabla f(x_t)\|^2 \leq \sum_{t=0}^{T-1} (f(x_t) - f(x_{t+1})) = f(x_0) - f(x_T).$$

Con $\gamma = \frac{1}{L}$, tenemos

$$\sum_{t=0}^{T-1} (f(x_t) - f(x^*)) \leq \frac{1}{2L} \sum_{t=0}^{T-1} \|\nabla f(x_t)\|^2 + \frac{L}{2} \|x_0 - x^*\|^2 \leq f(x_0) - f(x_T) + \frac{L}{2} \|x_0 - x^*\|^2,$$

equivalentemente,

$$\sum_{t=1}^T (f(x_t) - f(x^*)) \leq \frac{L}{2} \|x_0 - x^*\|^2.$$

Como $f(x_{t+1}) \leq f(x_t)$ para cada $0 \leq t \leq T$, por el lema 2.5,

$$f(x_T) - f(x^*) \leq \frac{1}{T} \sum_{t=1}^T (f(x_t) - f(x^*)) \leq \frac{L}{2T} \|x_0 - x^*\|^2.$$

□

Como hemos adelantado antes, esto mejora la cota del caso de gradiente acotado, ya que tomando $R^2 = \|x_0 - x^*\|^2$, antes necesitaba $T \geq \frac{R^2 L}{2\varepsilon}$ iteraciones en vez de $\frac{R^2 B^2}{\varepsilon^2}$ para alcanzar un error absoluto de, a lo sumo, ε (aunque es necesario aclarar que B y L no son el mismo valor).

Esto quiere decir que hemos pasado de una tasa sublineal lenta a una sublineal rápida. En términos prácticos esto quiere decir que para reducir el error a la décima parte (mismo ejemplo que hemos planteado en la sección anterior), ahora solo requerimos 10 veces más iteraciones, en lugar de las 100 que necesitábamos bajo la hipótesis de gradientes acotados. Esta ganancia en eficiencia explica por qué en la práctica se busca trabajar con funciones suaves o suavizar aquellas que no lo son mediante técnicas de regularización.

2. Descenso por gradiente en funciones convexas

2.1.5.2. Funciones fuertemente convexas

La función modelo $f(x) = x^2$ no es solo suave, sino que también es fuertemente convexa. Veamos este nuevo concepto, que hace que el algoritmo de descenso por gradiente sea aún más rápido.

Definición 2.7. Sea $f: \text{dom}(f) \rightarrow \mathbb{R}$ una función convexa y diferenciable, $X \subseteq \text{dom}(f)$ convexo y $\mu \in \mathbb{R}_+$, $\mu > 0$. La función f es llamada **fuertemente convexa** (de parámetro μ) sobre X si

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2} \|x - y\|^2, \quad \forall x, y \in X$$

Si $X = \text{dom}(f)$, f es llamada fuertemente convexa.

Esta definición implica que el gráfico de f está por encima de un paraboloide tangencial *no demasiado plano* en $(x, f(x))$, como se ve en la Figura 2.2.

Así, la gráfica de una función suave y fuertemente convexa está, en cada punto, encajada entre dos paraboloides.

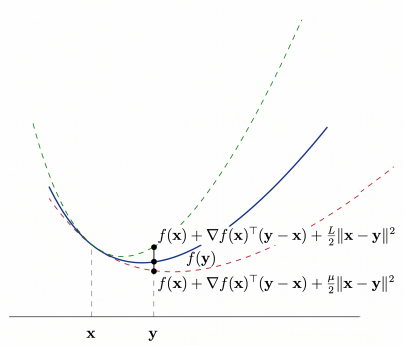


Figura 2.2.: Función fuertemente convexa (imagen generada con Gemini).

Tal y como hicimos con las funciones suaves (lema 2.4), podemos caracterizar las funciones fuertemente convexas con otra función:

Lema 2.8. Supongamos que $\text{dom}(f)$ es abierto y convexo y que $f: \text{dom}(f) \rightarrow \mathbb{R}$ es diferenciable. Equivalen:

1. f es fuertemente convexa de parámetro μ .
2. g definida por $g(x) = f(x) - \frac{\mu}{2} x^\top x$ es convexa sobre $\text{dom}(g) = \text{dom}(f)$.

Lema 2.9. Si $f: \mathbb{R}^d \rightarrow \mathbb{R}$ es fuertemente convexa de parámetro $\mu > 0$, entonces f es estrictamente convexa y tiene un único mínimo global.

Ahora que tenemos un único mínimo global x^* , queremos probar que $\lim_{t \rightarrow \infty} x_t = x^*$ en el descenso por gradiente.

Empezaremos haciendo el análisis convencional y pasando por límites inferiores tendremos que

$$g_t^\top (x_t - x^*) = \nabla f(x_t)^\top (x_t - x^*) \geq f(x_t) - f(x^*) + \frac{\mu}{2} \|x_t - x^*\|^2$$

debido a la fuerte convexidad.

Obtenemos

$$f(x_t) - f(x^*) \leq \frac{1}{2\gamma}(\gamma^2 \|\nabla f(x_t)\|^2 + \|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2) - \frac{\mu}{2} \|x_t - x^*\|^2.$$

Reescribir esto produce un límite inferior para $\|x_{t+1} - x^*\|^2$ en términos de $\|x_t - x^*\|^2$, junto con algunos términos sobrantes de los que luego nos encargaremos:

$$\|x_{t+1} - x^*\|^2 \leq 2\gamma(f(x^*) - f(x_t)) + \gamma^2 \|\nabla f(x_t)\|^2 + (1 - \mu\gamma) \|x_t - x^*\|^2.$$

Teorema 2.10. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa y diferenciable. Supongamos que f es suave de parámetro L y fuertemente convexa con parámetro $\mu > 0$. Tomando $\gamma = \frac{1}{L}$, el descenso por gradiente para un cierto x_0 arbitrario satisface:

1. Las distancias al cuadrado de x^* decrecen geométricamente:

$$\|x_{t+1} - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right) \|x_t - x^*\|^2, \quad t \geq 0.$$

2. El error absoluto tras T iteraciones es exponencialmente pequeño en T :

$$f(x_T) - f(x^*) \leq \frac{L}{2} \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2, \quad T > 0.$$

Demostración. 1. Por el descenso suficiente (lema 2.5), sabemos que

$$f(x^*) - f(x_t) \leq f(x_{t+1}) - f(x_t) \leq -\frac{1}{2L} \|\nabla f(x_t)\|^2.$$

Usando $\gamma = \frac{1}{L}$, multiplicando y dividiendo por 2γ y reordenando los términos obtenemos

$$2\gamma(f(x^*) - f(x_t)) + \gamma^2 \|\nabla f(x_t)\|^2 \leq 0.$$

De esto sigue

$$\|x_{t+1} - x^*\|^2 \leq (1 - \mu\gamma) \|x_t - x^*\|^2 = \left(1 - \frac{\mu}{L}\right) \|x_t - x^*\|^2$$

y

$$\|x_T - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2.$$

2. El resultado sigue del concepto de suavidad, usando que $\nabla f(x^*) = 0$:

$$f(x_T) - f(x^*) \leq \nabla f(x^*)^\top (x_T - x^*) + \frac{L}{2} \|x_T - x^*\|^2 = \frac{L}{2} \|x_T - x^*\|^2.$$

□

Derivando términos en el número de pasos necesarios (T) y usando la desigualdad de los logaritmos neperianos $\ln(1+x) \leq x$, se sigue que tras

$$T \geq \frac{L}{\mu} \ln \left(\frac{R^2 L}{2\varepsilon} \right)$$

2. Descenso por gradiente en funciones convexas

iteraciones tenemos un error absoluto de como máximo ε y como L, μ, R son constantes, esto nos da $\mathcal{O}(\log(1/\varepsilon))$.

El cociente $\kappa = L/\mu$ se conoce como el **número de condición de la función**. En el aprendizaje automático, este valor determina la dificultad del entrenamiento: si κ es pequeño (cercano a 1), el problema está bien condicionado y el descenso es directo; si κ es muy grande, las curvas de nivel de la función de pérdida son elipses muy alargadas, lo que provoca que el gradiente oscile y la convergencia sea mucho más lenta, a pesar de seguir siendo lineal.

2.1.6. Aceleración: El método de Nesterov

Aunque el descenso por gradiente es el estándar para minimizar funciones convexas diferenciables, no es el método más eficiente dentro de los algoritmos de primer orden. Mientras que el descenso por gradiente tradicional alcanza un error de $\mathcal{O}(1/T)$ tras T iteraciones, la teoría de complejidad establece un límite inferior óptimo de $\Omega(1/T^2)$ para esta clase de funciones, aunque el GD nunca alcanza ese límite (esto se verá mejor en la sección 2.4).

Esta brecha entre teoría y realidad se cierra con el *Algoritmo de Gradiente Acelerado de Nesterov* [9], un método que, aunque es menos intuitivo en su construcción, logra alcanzar matemáticamente dicho límite de eficiencia.

Este nuevo método se considera óptimo, ya que demuestra que es posible minimizar funciones suaves con una convergencia superior a la del gradiente tradicional usando la misma información inicial.

Ahora, presentaremos el algoritmo y haremos una breve demostración de que, en efecto, el algoritmo es óptimo.

Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa, diferenciable y suave de parámetro L . Partiendo de puntos iniciales $x_0 = y_0 = z_0$, el **algoritmo de gradiente acelerado** se define para cada iteración $t \geq 0$ como:

1. Paso de gradiente local:

$$y_{t+1} = x_t - \frac{1}{L} \nabla f(x_t). \quad (2.5)$$

2. Actualización del acumulador de dirección:

$$z_{t+1} = z_t - \frac{t+1}{2L} \nabla f(x_t). \quad (2.6)$$

3. Combinación de aceleración:

$$x_{t+1} = \frac{t+1}{t+3} y_{t+1} + \frac{2}{t+3} z_{t+1}. \quad (2.7)$$

Esto significa que estamos haciendo un *paso suave* normal desde x_t para obtener y_{t+1} y un *paso más agresivo* desde z_t para obtener z_{t+1} . El siguiente punto de iteración x_{t+1} es un promedio ponderado de y_{t+1} y z_{t+1} donde compensamos el paso más agresivo dándole a z_{t+1} un peso relativamente bajo.

Teorema 2.11. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa y diferenciable con mínimo global x^* . Supongamos además que f es suave con parámetro L .

El algoritmo de gradiente acelerado resulta

$$f(y_T) - f(x^*) \leq \frac{2L\|z_0 - x^*\|^2}{T(T+1)}, \quad T > 0.$$

Demostración. Para demostrar este resultado, haremos uso de la función potencial (o Función de Lyapunov) $\Phi(t)$, dada por

$$\Phi(t) = \underbrace{t(t+1)(f(y_t) - f(x^*))}_{\text{término de error}} + \underbrace{2L\|z_t - x^*\|^2}_{\text{término de distancia}},$$

Esta función combina el error de la función y la distancia al objetivo, veremos que $\Phi(t+1) \leq \Phi(t)$.

Si demostramos que la función potencial es decreciente, el término de error tiene que ser menor al valor inicial $\Phi(0)$ dividido por $T(T+1)$ y como $\Phi(0) = 2L\|z_0 - x^*\|^2$, obtenemos justo nuestro resultado a probar:

$$\underbrace{T(T+1)(f(y_T) - f(x^*)) + 2L\|z_T - x^*\|^2}_{\Phi(T)} \leq \underbrace{2L\|z_0 - x^*\|^2}_{\Phi(0)}.$$

Para ello, necesitamos hacer uso de tres resultados que ya conocemos:

1. Descenso suficiente (lema 2.5) para el paso (2.5) con $\gamma = \frac{1}{L}$:

$$f(y_{t+1}) \leq f(x_t) - \frac{1}{2L}\|\nabla f(x_t)\|^2. \quad (2.8)$$

2. Análisis estándar para el paso (2.6) con $\gamma = \frac{t+1}{2L}$ y $g_t = \nabla f(x_t)$:

$$g_t^\top(z_t - x^*) = \frac{t+1}{4L}\|g_t\|^2 + \frac{L}{t+1}(\|z_t - x^*\|^2 - \|z_{t+1} - x^*\|^2) \quad (2.9)$$

3. Convexidad:

$$f(x_t) - f(w) \leq g_t^\top(x_t - w), \quad w \in \mathbb{R}^d \quad (2.10)$$

Además de esto, hacemos unos cálculos más simples con el potencial:

$$\begin{aligned} \Phi(t+1) &= t(t+1)(f(y_{t+1}) - f(x^*)) + 2(t+1)(f(y_{t+1}) - f(x^*)) + 2L\|z_{t+1} - x^*\|^2 \\ \Phi(t) &= t(t+1)(f(y_t) - f(x^*)) + 2L\|z_t - x^*\|^2 \end{aligned}$$

Ahora, consideramos

$$\Delta = \frac{\Phi(t+1) - \Phi(t)}{t+1}$$

y nuestro objetivo pasa a ser demostrar que $\Delta \leq 0$.

2. Descenso por gradiente en funciones convexas

Δ puede ser expresado como sigue:

$$\begin{aligned}
\Delta &= t(f(y_{t+1}) - f(y_t)) + 2(f(y_{t+1}) - f(x^*)) + \frac{2L}{t+1} (\|z_{t+1} - x^*\|^2 - \|z_t - x^*\|^2) \\
&\stackrel{(2.9)}{=} t(f(y_{t+1}) - f(y_t)) + 2(f(y_{t+1}) - f(x^*)) + \frac{t+1}{2L} \|g_t\|^2 - 2g_t^\top (z_t - x^*) \\
&\stackrel{(2.8)}{\leq} t(f(x_t) - f(y_t)) + 2(f(x_t) - f(x^*)) - \frac{1}{2L} \|g_t\|^2 - 2g_t^\top (z_t - x^*) \\
&\leq t(f(x_t) - f(y_t)) + 2(f(x_t) - f(x^*)) - 2g_t^\top (z_t - x^*) \\
&\stackrel{(2.10)}{\leq} tg_t^\top (x_t - y_t) + 2g_t^\top (x_t - x^*) - 2g_t^\top (z_t - x^*) \\
&= g_t^\top ((t+2)x_t - ty_t - 2z_t) \\
&\stackrel{(2.7)}{=} g_t^\top \mathbf{0} = 0
\end{aligned}$$

Lo que implica que $\Phi(t+1) \leq \Phi(t)$. □

Esto garantiza que el error $f(y_T) - f(x^*)$ debe encogerse a una velocidad de al menos $1/T(T+1)$ para compensar el crecimiento de los coeficientes temporales en la función potencial.

Comparando con el teorema 2.6, vemos que el orden es $\mathcal{O}(1/T^2)$ en vez de $\mathcal{O}(1/T)$, por lo que para un error máximo ε necesitamos solo $\mathcal{O}(1/\sqrt{\varepsilon})$ pasos en vez de $\mathcal{O}(1/\varepsilon)$.

Esta reducción es crucial en problemas de gran escala; la capacidad de reducir el número de iteraciones de $1/\varepsilon$ a $1/\sqrt{\varepsilon}$ permite que modelos que tardarían semanas en converger puedan ser entrenados en cuestión de días.

Observación 2.12. A diferencia del descenso por gradiente clásico, muchos métodos acelerados incorporan un término de «momentum», que introduce dependencia respecto a iteraciones anteriores. Intuitivamente, esto permite acumular velocidad acelerando la convergencia. Sin embargo, esta inercia también puede provocar oscilaciones cerca del óptimo, fenómeno característico de métodos acelerados como el de Nesterov [9].

2.2. Descenso por subgradiente

Hasta ahora hemos asumido que el gradiente existe en todo el dominio. Sin embargo, en el aprendizaje automático es común usar funciones no diferenciables que presentan «picos» donde el gradiente no está definido.

Para optimizar estos modelos, generalizamos el concepto de gradiente mediante el **subgradiente**. Aunque este método sacrifica la rapidez de las tasas aceleradas y no garantiza un descenso en cada iteración, permite asegurar la convergencia en superficies convexas irregulares donde los métodos clásicos fallarían.

2.2.1. El concepto de subdiferenciabilidad

Definición 2.13. Sea $f: \text{dom}(f) \rightarrow \mathbb{R}$, diremos que $g \in \mathbb{R}^d$ es un **subgradiente** de f en $x \in \text{dom}(f)$ si

$$f(y) \geq f(x) + g^\top (y - x) \quad \forall y \in \text{dom}(f).$$

El conjunto de subgradientes de f en x es llamado el **subdiferencial en x** y es denotado por $\partial f(x)$.

El concepto de subgradiente puede verse como una generalización del gradiente para funciones que no son necesariamente diferenciables.

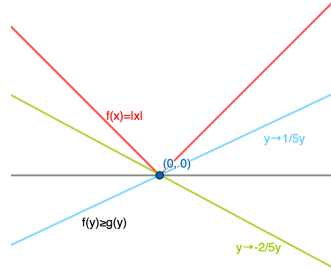


Figura 2.3.: La función $f(x) = |x|$ tiene subgradientes $g \in [-1, 1]$ en 0, ya que $f(y) \geq gy$ para todo $g \in [-1, 1]$ (imagen de creación propia usando GeoGebra).

Geométricamente, cada subgradiente define un hiperplano de soporte que toca la gráfica de la función en x pero se mantiene por debajo de ella.

Esto quiere decir que en el caso diferenciable, existe exactamente un subgradiente $\nabla f(x)$, o ninguno.

Sin embargo, en puntos «angulosos» como el origen en $f(x) = |x|$, existen infinitos subgradientes (en este caso concreto, el intervalo $[-1, 1]$), tal y como se ve en la Figura 2.3.

Podemos notar que la definición de subgradiente es bastante similar a la caracterización de primer orden de la convexidad (lema 1.7), de hecho, la única diferencia es que hemos cambiado $\nabla f(x)$ por g .

Por tanto, podríamos decir que la convexidad es equivalente a la existencia de subgradientes en todos los puntos, lo que nos lleva a una «primera caracterización de la convexidad» que cubre también el caso no diferenciable.

Lema 2.14. Si una función $f: \text{dom}(f) \rightarrow \mathbb{R}$ es convexa y su dominio es abierto, entonces $\partial f(x) \neq \emptyset \quad \forall x \in \text{dom}(f)$.

De forma similar, podemos caracterizar la continuidad de Lipschitz por subgradientes.

Lema 2.15. Sea $f: \text{dom}(f) \rightarrow \mathbb{R}$ convexa, $\text{dom}(f)$ abierto, $B \in \mathbb{R}^+$. Las siguientes afirmaciones son equivalentes:

- (I) $\|g\| \leq B \quad \forall x \in \text{dom}(f) \text{ y } \forall g \in \partial f(x)$.
- (II) $|f(x) - f(y)| \leq B\|x - y\| \quad \forall x, y \in \text{dom}(f)$.

El concepto del subgradiente también nos permite descubrir casos de optimización para funciones que no son necesariamente diferenciables (y no necesariamente convexas).

Lema 2.16. Sea $f: \text{dom}(f) \rightarrow \mathbb{R}$ y $x \in \text{dom}(f)$. Si $0 \in \partial f(x)$, entonces x es un mínimo global.

Demostración. Si

$$g = 0 \in \partial f(x) \Rightarrow f(y) \geq f(x) + g^\top(y - x) = f(x) \quad \forall y \in \text{dom}(f).$$

2. Descenso por gradiente en funciones convexas

Por tanto, x es un mínimo global. $\square$

Como podemos ver, el concepto de subgradiente es más fuerte que el de gradiente para funciones diferenciables. De hecho, si $\nabla f(x) = 0$ para una función diferenciable f y $x \in \text{dom}(f)$, solo podemos saber que x es un punto crítico, pero no necesariamente que sea un mínimo global.

2.2.2. Dinámica del algoritmo

A diferencia del gradiente, el subgradiente no garantiza que la dirección $-g_t$ reduzca el valor de la función en cada iteración, lo que llevado a la práctica puede traducirse en ciertas oscilaciones. Es decir, es posible que $f(x_{t+1}) > f(x_t)$. Sin embargo, lo que sí se puede demostrar es que, con un tamaño de paso adecuado, la distancia al óptimo $\|x_t - x^*\|$ se puede acotar y obtener garantías para promedios.

De esta forma, una iteración del descenso por subgradiente es definida como:

$$\begin{aligned} \text{Sea } g_t &\in \partial f(x_t) \\ x_{t+1} &= x_t - \gamma_t g_t. \end{aligned}$$

En contraste con nuestro anterior algoritmo de descenso, en este caso el tamaño de paso depende del tiempo.

2.2.2.1. Convergencia en escenarios de curvatura acotada

El siguiente resultado nos habla de la convergencia para el descenso por subgradiente. Es idéntico al teorema 2.2, pero sin la hipótesis de diferenciabilidad.

Teorema 2.17. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa y B -Lipschitz continua con un mínimo global x^* , además supongamos que $\|x_0 - x^*\| \leq R$. Eligiendo $\gamma_t = \gamma = \frac{R}{B\sqrt{T}}$ como constante de paso, el descenso por subgradiente queda

$$\frac{1}{T} \sum_{t=0}^{T-1} f(x_t) - f(x^*) \leq \frac{RB}{\sqrt{T}}.$$

Demostración. La prueba es igual a la del teorema 2.2.

El único cambio es que g_t es ahora un subgradiente y que por tanto la desigualdad (2.2) se da ahora por la propia definición de subgradiente en vez de por la caracterización de primer orden de la convexidad.

La desigualdad $\|g_t\|^2 \leq B^2$ sigue del lema 2.15. $\square$

Observación 2.18. En métodos de subgradiente, las iteraciones individuales pueden presentar oscilaciones significativas debido a la falta de diferenciabilidad de la función objetivo. Por esta razón, es habitual considerar promedios de las iteraciones,

$$\bar{x}_T = \frac{1}{T} \sum_{t=1}^T x_t,$$

ya que estos suelen mostrar un comportamiento más estable y mejores garantías teóricas de convergencia.

2.2.2.2. Convergencia en escenarios de suavidad y fuerte convexidad

Aunque el escenario de suavidad permite alcanzar tasas de $\mathcal{O}(1/\varepsilon)$ tal y como hemos visto anteriormente, en funciones no diferenciables la fuerte convexidad por sí sola no garantiza la convergencia. La ausencia de una cota superior cuadrática permite crecimientos explosivos (como podemos ver en el caso de $f(x) = e^{|x|}$) donde el algoritmo puede divergir por un efecto de *overshoot* incluso con pasos minúsculos.

Para recuperar el control del algoritmo sin recurrir a la suavidad, es necesario imponer que el subdiferencial esté acotado ($\|g\| \leq B$). Bajo esta hipótesis, la fuerte convexidad actúa como un acelerador que permite pasar de la tasa estándar de $\mathcal{O}(1/\varepsilon^2)$ a una tasa de $\mathcal{O}(1/\varepsilon)$. Aunque este ritmo es inferior al caso diferenciable, representa una mejora en la eficiencia para funciones con irregularidades. A continuación, formalizamos este resultado:

Teorema 2.19. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función fuertemente convexa con parámetro $\mu > 0$ y sea x^* el único mínimo global de f . Con tamaño de paso decreciente

$$\gamma_t = \frac{2}{\mu(t+1)}, \quad t > 0$$

el descenso por subgradiente sigue

$$f\left(\frac{2}{T(T+1)} \sum_{t=1}^T tx_t\right) - f(x^*) \leq \frac{2B^2}{\mu(T+1)}$$

donde $B = \max_{t=1}^T \|g_t\|$.

Demostración. Empezamos por el análisis estándar (con $\gamma = \gamma_t$):

$$g_t^\top (x_t - x^*) = \frac{\gamma_t}{2} \|g_t\|^2 + \frac{1}{2\gamma_t} (\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2).$$

Ahora, tomamos el límite inferior que proporciona la definición de convexidad fuerte

$$g_t^\top (x_t - x^*) \geq f(x_t) - f(x^*) + \frac{\mu}{2} \|x_t - x^*\|^2$$

y como resultado de la convexidad fuerte (con $\|g_t\|^2 \leq B^2$) tenemos que

$$f(x_t) - f(x^*) \leq \frac{B^2\gamma_t}{2} + \frac{(\gamma_t^{-1} - \mu)}{2} \|x_t - x^*\|^2 - \frac{\gamma_t^{-1}}{2} \|x_{t+1} - x^*\|^2. \quad (2.11)$$

A diferencia del análisis estándar en el que teníamos $\gamma_t = \gamma$ y $\mu = 0$, el lado derecho de la desigualdad no sigue una serie telescópica para $t \leq T$.

Para ajustar esto es para lo que necesitamos que el tamaño de paso varíe.

Hagamos un pequeño cálculo: para obtener un comportamiento telescópico, necesitaríamos que $\gamma_t^{-1} = \gamma_{t+1}^{-1} - \mu$.

Por ejemplo, $\gamma_t^{-1} = \mu(1+t)$ satisface esto, pero nuestra elección $\gamma_t^{-1} = \frac{\mu(t+1)}{2}$ no. Sin embargo, demostraremos que esta elección permite una estructura de suma telescópica ponderada que optimiza la cota final.

Al aplicar este tamaño de paso y multiplicar por t ambos lados obtenemos, se observa que

2. Descenso por gradiente en funciones convexas

con esta elección obtenemos una ponderación temporal:

$$\begin{aligned} t(f(x_t) - f(x^*)) &\leq \frac{B^2 t}{\mu(t+1)} + \frac{\mu}{4}(t(t-1)\|x_t - x^*\|^2 - (t+1)t\|x_{t+1} - x^*\|^2) \\ &\leq \frac{B^2}{\mu} + \frac{\mu}{4}(t(t-1)\|x_t - x^*\|^2 - (t+1)t\|x_{t+1} - x^*\|^2). \end{aligned}$$

Lo que logramos con este ajuste es inducir una estructura telescópica de segundo orden. Ya no solo cancelamos distancias al cuadrado, sino los términos escalados por los coeficientes cuadrados $t(t-1)$ y $(t+1)t$.

Sumando $t = 1, \dots, T$, obtenemos la suma telescópica

$$\sum_{t=1}^T t(f(x_t) - f(x^*)) \leq \frac{TB^2}{\mu} + \frac{\mu}{4}(0 - T(T+1)\|x_{T+1} - x^*\|^2) \leq \frac{TB^2}{\mu}.$$

Como

$$\frac{2}{T(T+1)} \sum_{t=1}^T t = 1,$$

la desigualdad de Jensen (Lema 1.3) nos da

$$f\left(\frac{2}{T(T+1)} \sum_{t=1}^T t(x_t)\right) - f(x^*) \leq \frac{2}{T(T+1)} \sum_{t=1}^T t(f(x_t) - f(x^*)),$$

lo que implica

$$f\left(\frac{2}{T(T+1)} \sum_{t=1}^T t(x_t)\right) - f(x^*) \leq \frac{2B^2}{\mu(T+1)}.$$

□

Al utilizar este tamaño de paso, como la gran mayoría de los componentes se han anulado de forma consecutiva al realizar la suma telescópica, la cota superior que obtendremos estará mucho más ajustada para el error promedio.

De hecho, es fácil ver que, tal y como habíamos predicho, el orden de convergencia es $\mathcal{O}(1/\varepsilon)$. Simplemente igualando esta cota a ε y despejando T , obtenemos:

$$T \geq \frac{2B^2}{\mu\varepsilon} - 1,$$

lo que confirma que el número de iteraciones necesarias escala como $\mathcal{O}(1/\varepsilon)$. Este resultado es significativamente más eficiente que el $\mathcal{O}(1/\varepsilon^2)$ de las funciones convexas generales, demostrando el poder acelerador de la fuerte convexidad incluso en ausencia de suavidad.

2.3. Descenso por gradiente estocástico

Hasta ahora, hemos asumido que en cada iteración disponemos del gradiente (o subgradiente) exacto de la función objetivo.

En el contexto del aprendizaje automático, la función de pérdida suele ser un promedio

sobre un conjunto de entrenamiento de n muestras:

$$f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x) \quad (2.12)$$

donde f_i es la función de coste del i -ésimo dato, tomando un conjunto de entrenamiento de n elementos en total.

El factor de normalización $1/n$ nos ayuda a simplificar los cálculos que haremos próximamente.

Cuando n es demasiado grande, cuesta mucho calcular $\nabla f(x)$. El descenso por gradiente estocástico (SGD) rompe esta barrera sustituyendo el gradiente total por una estimación basada en una única muestra (o un pequeño grupo de ellas) elegida al azar.

Una iteración del **descenso por gradiente estocástico (SGD)** [2] está básicamente definida como:

$$\begin{aligned} \text{muestra } [i] &\in n \text{ escogida aleatoriamente uniformemente} \\ x_{t+1} &= x_t - \gamma_t \nabla f_i(x_t). \end{aligned}$$

Aunque este vector $g_t = \nabla f_i(x_t)$ es una estimación con alta varianza que no garantiza el descenso en cada iteración individual, es n veces más eficiente por paso que el método tradicional. Debido a esta naturaleza aleatoria, el análisis de convergencia deja de ser determinista y requiere el uso de la esperanza matemática para demostrar que, en promedio, el algoritmo se dirige hacia el óptimo.

Al igual que en el caso del subgradiente, esta incertidumbre inherente obligará a emplear tamaños de paso variables (γ_t) para asegurar la estabilidad final del método.

La ventaja del SGD frente al descenso por gradiente normal es su eficiencia por iteración: mientras que para calcular el gradiente de una función del tipo (2.12) necesitaríamos calcular n gradientes, una iteración del SGD necesita solo una de ellas, luego es n veces más barato por iteración.

Por tanto, el SGD se ha convertido en el principal recurso para modelos de aprendizaje automático.

2.3.1. Insesgadez del gradiente estocástico

Comenzamos de nuevo con el análisis convencional, pero ahora no podemos acotar la variable aleatoria $g_t^\top (x_t - x^*)$, por lo que la desigualdad

$$f(x_t) - f(x^*) \leq g_t^\top (x_t - x^*)$$

puede cumplirse o no dependiendo de cómo sea g_t .

Sin embargo, haciendo uso de las esperanzas, vamos a ver que sí se cumple el resultado.

La validez del SGD reside en que, aunque el vector g_t es una aproximación ruidosa, actúa como estimador insesgado del gradiente real. Formalmente, esto quiere decir que la esperanza condicionada de g_t respecto a la trayectoria actual x_t coincide con el gradiente

2. Descenso por gradiente en funciones convexas

completo:

$$\mathbb{E}[g_t | x_t = x] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(x) = \nabla f(x), \quad x \in \mathbb{R}^d. \quad (2.13)$$

donde $\mathbb{E}[g_t | x_t = x]$ representa la esperanza condicionada de g_t dado el elemento $\{x_t = x\}$.

Gracias a la linealidad de las esperanzas condicionales, podemos proyectar esta propiedad sobre nuestro análisis de convergencia:

$$\mathbb{E}[g_t^\top (x - x^*) | x_t = x] = \mathbb{E}[g_t | x_t = x]^\top (x - x^*) = \nabla f(x)^\top (x - x^*).$$

Usando el hecho de que $\{x_t = x\}$ solo puede ocurrir para x en algún conjunto finito X (un elemento por cada elección de índices a lo largo de todas las iteraciones), el Teorema de la Partición nos da además:

$$\begin{aligned} \mathbb{E}[g_t^\top (x - x^*)] &= \sum_{x \in X} \mathbb{E}[g_t^\top (x - x^*) | x_t = x] P(x_t = x) \\ &= \sum_{x \in X} \nabla f(x)^\top (x - x^*) P(x_t = x) \\ &= \mathbb{E}[\nabla f(x_t)^\top (x_t - x^*)]. \end{aligned}$$

Por tanto tenemos

$$\mathbb{E}[g_t^\top (x_t - x^*)] = \mathbb{E}[\nabla f(x_t)^\top (x_t - x^*)] \geq \mathbb{E}[f(x_t) - f(x^*)] \quad (2.14)$$

La última desigualdad se debe a la convexidad y esto implica que el límite inferior (2.2) se cumple en esperanza.

Bajo (2.13) decimos que el gradiente estocástico es un **estimador insesgado** del gradiente para cualquier iteración t .

Este resultado es clave: demuestra que, aunque no podamos garantizar el descenso en cada paso individual, el algoritmo se comporta como el descenso por gradiente tradicional en esperanza.

2.3.2. Análisis de convergencia estocástica

Una vez demostrada la insesgadez de g_t , el análisis de convergencia se traslada al estudio de los valores esperados de la función de pérdida. Ahora, ya no buscamos un descenso monótono en cada paso, sino una tendencia asintótica hacia el óptimo.

A continuación formalizaremos cómo el SGD hereda las tasas de convergencia del caso determinista bajo la hipótesis de gradientes acotados en esperanza ($\mathbb{E}[\|g_t\|^2] \leq B^2$). Veremos que el algoritmo recupera la tasa de $\mathcal{O}(1/\sqrt{T})$ para funciones convexas y mejora a $\mathcal{O}(1/T)$ bajo convexidad fuerte, validando la eficiencia del método a pesar de la incertidumbre en la dirección de descenso.

Para obtener un primer resultado a partir del análisis convencional asumimos que el gradiente está acotado, $\|\nabla f(x)\|^2 \leq B^2$, donde B era una constante. Aquí, asumiremos lo mismo para las normas al cuadrado esperadas de nuestros gradientes estocásticos. Obtendremos el mismo resultado, excepto que ahora se cumple para los valores esperados de la función.

Teorema 2.20. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función convexa y diferenciable, y sea x^* un mínimo global de f ; además supongamos que

$$\|x_0 - x^*\| \leq R \quad \text{y que} \quad \mathbb{E}[\|g_t\|^2] \leq B^2 \quad \forall t.$$

Eligiendo el tamaño de paso de $\gamma = \frac{R}{B\sqrt{T}}$, el descenso por gradiente estocástico produce

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[f(x_t)] - f(x^*) \leq \frac{RB}{\sqrt{T}}.$$

Demostración. Tomando esperanzas en ambos lados del análisis convencional y usando la linealidad de la esperanza, obtenemos

$$\sum_{t=0}^{T-1} \mathbb{E}[g_t^\top (x_t - x^*)] \leq \frac{\gamma}{2} \sum_{t=0}^{T-1} \mathbb{E}[\|g_t\|^2] + \frac{1}{2\gamma} \|x_0 - x^*\|^2. \quad (2.15)$$

Por (2.14), se tiene

$$\mathbb{E}[f(x_t) - f(x^*)] \leq \mathbb{E}[g_t^\top (x_t - x^*)].$$

Sustituyendo esto en (2.15), usando $\mathbb{E}[\|g_t\|^2] \leq B^2$ y $\|x_0 - x^*\| \leq R$, tenemos

$$\sum_{t=0}^{T-1} \mathbb{E}[f(x_t) - f(x^*)] \leq \frac{\gamma}{2} B^2 T + \frac{1}{2\gamma} R^2,$$

lo cual demuestra el enunciado a partir de la elección de γ , como en el teorema 2.2. $\square$

Este resultado confirma una tasa de $\mathcal{O}(1/\varepsilon^2)$ pasos, idéntica a la del descenso por subgradiente, pero con un coste computacional por iteración significativamente menor.

Observación 2.21 (Fenómeno del *noise floor* [2]). A diferencia de cuando podemos tomar un tamaño de paso constante que nos garantice la convergencia hacia el óptimo, en el SGD el ruido introducido por el muestreo aleatorio hace que, cuando el algoritmo se encuentre próximo al óptimo x^* , el gradiente estocástico g_t no es necesariamente nulo, por lo que las iteraciones continúan fluctuando alrededor de la solución en lugar de converger exactamente hacia ella.

Este error residual es conocido como *noise floor* y depende principalmente de la varianza del estimador estocástico y del tamaño de paso utilizado.

Por esta razón, una estrategia clásica para garantizar convergencia exacta consiste en emplear tamaños de paso decrecientes que tiendan a 0. Alternativamente, más adelante veremos otras formas de reducir progresivamente la varianza del gradiente mediante el uso de estrategias como la utilización de *mini-batches* de gran tamaño.

Si añadimos la hipótesis de convexidad fuerte, la velocidad de convergencia mejora sustancialmente, permitiendo reducir el error de forma más agresiva mediante un tamaño de paso decreciente.

De nuevo, la demostración funciona tomando esperanzas sobre un análisis previo, en este caso, el del descenso por subgradiente en el caso fuertemente convexo (teorema 2.19).

Teorema 2.22. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ una función fuertemente convexa con parámetro $\mu > 0$, sea x^* el único mínimo global de f .

2. Descenso por gradiente en funciones convexas

Con un tamaño de paso decreciente $\gamma_t = \frac{2}{\mu(t+1)}$, el descenso por gradiente estocástico produce

$$\mathbb{E} \left[f \left(\frac{2}{T(T+1)} \sum_{t=1}^T tx_t \right) - f(x^*) \right] \leq \frac{2B^2}{\mu(T+1)}$$

donde $\mathbb{E}[\|g_t\|^2] \leq B^2 \quad \forall t$.

Demostración. Comenzamos desde el análisis estándar con $\gamma = \gamma_t$ y tomamos esperanzas en ambos lados

$$\mathbb{E}[g_t^\top (x_t - x^*)] = \frac{\gamma_t}{2} \mathbb{E}[\|g_t\|^2] + \frac{1}{2\gamma_t} (\mathbb{E}[\|x_t - x^*\|^2] - \mathbb{E}[\|x_{t+1} - x^*\|^2]).$$

Ahora, usamos (2.14) junto con la fuerte convexidad para obtener un límite inferior en el lado izquierdo:

$$\mathbb{E}[g_t^\top (x_t - x^*)] = \mathbb{E}[\nabla f(x_t)^\top (x_t - x^*)] \geq \mathbb{E}[f(x_t) - f(x^*)] + \frac{\mu}{2} \mathbb{E}[\|x_t - x^*\|^2].$$

Combinamos las dos ecuaciones anteriores y usando $\mathbb{E}[\|g_t\|^2] \leq B^2$ obtenemos la versión esperada de (2.11)

$$\mathbb{E}[f(x_t) - f(x^*)] \leq \frac{B^2\gamma_t}{2} + \frac{(\gamma_t^{-1} - \mu)}{2} \mathbb{E}[\|x_t - x^*\|^2] - \frac{\gamma_t^{-1}}{2} \mathbb{E}[\|x_{t+1} - x^*\|^2].$$

El resto de la demostración sigue igual que la del teorema 2.19, pero tomando esperanzas en cada paso. $\square$

Estos resultados validan al SGD como una herramienta robusta: el ruido introducido por el muestreo aleatorio no degrada la tasa de convergencia asintótica, siempre que el tamaño de paso se gestione adecuadamente.

2.3.3. Generalización al caso no diferenciable

El análisis anterior puede extenderse directamente a problemas que no son necesariamente diferenciables: modificamos el SGD para usar un subgradiente de f_i en cada iteración.

La actualización del **descenso por subgradiente estocástico** viene dada por:

seleccionar $i \in [n]$ uniformemente al azar

sea $g_t \in \partial f_i(x_t)$

$x_{t+1} = x_t - \gamma_t g_t$

La clave de esta generalización es que la propiedad de insesgadez se preserva: si cada $g_i(x)$ es un subgradiente de la función de coste individual f_i , entonces su promedio sobre todas las muestras es un subgradiente de la función objetivo total f .

Es decir:

$$\mathbb{E}[g_t | x_t = x] = \frac{1}{n} \sum_{i=1}^n g_i(x) \in \partial f(x).$$

Dado que la estructura de los límites inferiores de convexidad y fuerte convexidad se mantiene en esperanza, los resultados de convergencia de $\mathcal{O}(1/\varepsilon^2)$ y $\mathcal{O}(1/\varepsilon)$ pasos obtenidos

para el caso diferenciable son válidos también para el descenso por subgradiente estocástico.

Esta versatilidad permite aplicar el método a una amplia gama de problemas de optimización en aprendizaje automático donde la suavidad no está garantizada.

2.3.4. Estrategia de *mini-batches*

En la práctica, la variante más utilizada del SGD es el **descenso por gradiente por *mini-batches*** [2]. En lugar de calcular el gradiente para una sola muestra, se utiliza el promedio de un subconjunto de m elementos:

$$\tilde{g}_t = \frac{1}{m} \sum_{j=1}^m \nabla f_{i_j}(x_t).$$

Esta fórmula es muy flexible: si $m = 1$ recuperamos el SGD puro, mientras que con $m = n$ obtenemos el descenso por gradiente usual (Batch).

El uso de *mini-batches* ofrece grandes ventajas tanto desde el punto de vista computacional como matemático:

1. Paralelización y eficacia de hardware El cálculo de $\tilde{g}_t$ es el coste principal del algoritmo.

Al definirse todos los componentes ∇f_{i_j} sobre el mismo punto x_t , estos pueden calcularse de forma independiente. Esta propiedad permite la paralelización en múltiples procesadores o GPUs, aprovechando que las arquitecturas modernas están optimizadas para operaciones matriciales masivas.

Para entender por qué usamos *mini-batches*, imagina que tienes que corregir 100 exámenes:

- El gradiente estocástico ($m=1$) sería como corregirlos uno a uno, terminando uno antes de empezar el siguiente.
- El *mini-batch* ($m=64$) es como si tuvieras a 64 profesores trabajando a la vez en la misma mesa, pudiendo así corregir 64 exámenes a la vez.

Las tarjetas gráficas (GPUs) de los ordenadores modernos funcionan exactamente así: no son un único procesador muy rápido, sino miles de procesadores pequeños trabajando en paralelo. Calcular el gradiente de 64 datos a la vez no tarda 64 veces más que calcular uno solo; gracias a esta arquitectura, el ordenador tarda prácticamente lo mismo. Por tanto, el *mini-batch* nos permite procesar muchísima más información en el mismo tiempo.

2. Reducción de la varianza Desde una perspectiva probabilística, tomar el promedio de variables aleatorias independientes reduce el ruido de la estimación.

Como se observa en el siguiente desarrollo, la varianza de nuestra estimación $\tilde{g}_t$ respecto al gradiente real disminuye proporcionalmente al tamaño del *mini-batch* m :

$$\mathbb{E}[\|\tilde{g}_t - \nabla f(x_t)\|^2] = \mathbb{E}\left[\left\|\frac{1}{m} \sum_{j=1}^m g_t^j - \nabla f(x_t)\right\|^2\right] = \frac{1}{m} \mathbb{E}[\|g_t^1 - \nabla f(x_t)\|^2] \leq \frac{B^2}{m}.$$

Esta propiedad hace que la trayectoria del algoritmo sea mucho más estable y tenga menos errores que en el caso estocástico simple, facilitando una convergencia más fluida hacia el óptimo.

2. Descenso por gradiente en funciones convexas

3. Regularización implícita. A pesar de la reducción de varianza, el ruido implícito en el mini-batch actúa como un regularizador natural. Este comportamiento ayuda al algoritmo a «saltar» mínimos locales poco profundos y puntos de silla, favoreciendo la convergencia hacia zonas del espacio de parámetros con mejor capacidad de generalización en modelos de aprendizaje profundo.

2.4. Optimalidad de los métodos de primer orden

Tras haber analizado diversas variantes del descenso por gradiente y sus respectivas tasas de convergencia, es natural preguntarnos si los resultados obtenidos son los mejores posibles.

Sorprendentemente, en general sí lo son. Este hecho se formaliza en el siguiente teorema:

Teorema 2.23 (Nesterov). *Para cualquier $T \leq d - 1$ y todo punto de partida x_0 , existe una función f en la clase de funciones B -Lipschitz sobre $\mathbb{R}^d$, tal que cualquier método de (sub)gradiente tiene un error objetivo al menos de*

$$f(x_T) - f(x^*) \geq \frac{RB}{2(1 + \sqrt{T+1})}.$$

Este teorema establece un límite teórico para la optimización: para cualquier método que genere sus iteraciones como combinaciones lineales de (sub)gradientes pasados, existe siempre una función dentro de la clase de funciones convexas B -Lipschitz tal que el error objetivo es, al menos, de orden $\Omega(1/\sqrt{T})$. Solo mediante la introducción de hipótesis adicionales, como la suavidad o la fuerte convexidad, somos capaces de superar esta barrera y alcanzar las velocidades que hoy permiten entrenar los modelos más avanzados de aprendizaje automático, como recoge la Tabla 2.2.

Clase de funciones	Cota inferior	Algoritmo estudiado	Iteraciones
Convexas B -Lipschitz	$\Omega(1/\sqrt{T})$	(Sub)gradiente	$\mathcal{O}(1/\varepsilon^2)$
Convexas suaves	$\Omega(1/T^2)$	Gradiente acelerado	$\mathcal{O}(1/\sqrt{\varepsilon})$
Fuertemente convexas	$\Omega(1/T)$	Subgradiente	$\mathcal{O}(1/\varepsilon)$
Fuert. convexas y suaves	$\Omega((1 - \mu/L)^T)$	Gradiente, $\gamma = 1/L$	$\mathcal{O}(\log(1/\varepsilon))$
Convexas no dif.	$\Omega(1/\sqrt{T})$	Subgradiente	$\mathcal{O}(1/\varepsilon^2)$

Tabla 2.2.: Tasas óptimas de convergencia para métodos de primer orden según la clase de función.

La velocidad de convergencia alcanzable depende, por tanto, de las propiedades geométricas de la función objetivo, no del algoritmo elegido dentro de esta clase.

En definitiva, la optimización moderna no busca el *algoritmo perfecto*, sino el equilibrio ideal entre la suavidad de la función, la curvatura del espacio y el volumen de datos disponible.

3. Descenso por gradiente en funciones no convexas

La mayoría de los resultados vistos hasta ahora garantizan la convergencia al mínimo global bajo supuestos de convexidad. Sin embargo, muchas funciones de pérdida relevantes en aprendizaje automático (en particular en contextos de redes neuronales y otras aplicaciones presentes en las tecnologías actuales) no son convexas. En este capítulo analizaremos en qué medida el descenso por gradiente sigue siendo útil en este contexto más general.

El obstáculo fundamental es el siguiente: en funciones no convexas, el algoritmo puede quedar atrapado en mínimos locales, puntos de silla o incluso no converger a ningún punto (Figura 3.1). Aun así, en condiciones favorables es posible establecer resultados de convergencia como los que veremos a continuación.

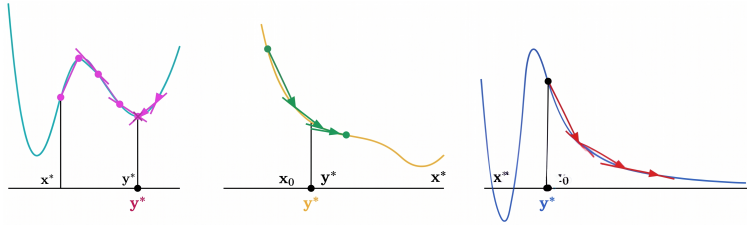


Figura 3.1.: El descenso por gradiente puede converger a un mínimo local $y^* \neq x^*$ (imagen izquierda), a un punto de silla y^* (imagen central) o incluso a nada (imagen derecha) (imagen generada con Gemini).

3.1. Funciones suaves no convexas

En el caso convexo, la suavidad combinada con el descenso suficiente permitía controlar el error $f(x_t) - f(x^*)$. En el caso no convexo, dicha diferencia ya no es un indicador fiable de progreso, pero la norma del gradiente $\|\nabla f(x_t)\|^2$ sí puede acotarse.

Teorema 3.1. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ diferenciable con mínimo global x^* , y suave con parámetro L . Elijiendo $\gamma = \frac{1}{L}$, el descenso por gradiente satisface

$$\frac{1}{T} \sum_{t=0}^{T-1} \|\nabla f(x_t)\|^2 \leq \frac{2L}{T} (f(x_0) - f(x^*)), \quad T > 0.$$

En particular, $\|\nabla f(x_t)\|^2 \leq \frac{2L}{T} (f(x_0) - f(x^*))$ para algún $t \in \{0, \dots, T-1\}$ y $\|\nabla f(x_t)\|^2 \rightarrow 0$ cuando $t \rightarrow \infty$.

Demostración. Del lema de descenso suficiente (lema 2.5), que no requiere convexidad, se tiene

$$f(x_{t+1}) \leq f(x_t) - \frac{1}{2L} \|\nabla f(x_t)\|^2.$$

3. Descenso por gradiente en funciones no convexas

Reordenando, $\|\nabla f(x_t)\|^2 \leq 2L(f(x_t) - f(x_{t+1}))$. Sumando sobre $t = 0, \dots, T-1$ se obtiene una suma telescópica:

$$\sum_{t=0}^{T-1} \|\nabla f(x_t)\|^2 \leq 2L(f(x_0) - f(x_T)) \leq 2L(f(x_0) - f(x^*)).$$

Dividiendo por T se concluye. $\square$

Este resultado garantiza que el promedio de las normas al cuadrado del gradiente converge a 0 a una tasa $\mathcal{O}(1/T)$, es decir, existe algún $t \in \{0, \dots, T-1\}$ con $\|\nabla f(x_t)\|^2 \leq \frac{2L}{T}(f(x_0) - f(x^*))$. Sin embargo, la convergencia de $\|\nabla f(x_t)\|^2$ a 0 no implica en general la convergencia a un punto crítico: el algoritmo puede entrar en una región en la que f se aproxima asintóticamente a cierto valor sin alcanzarlo nunca, con el gradiente tendiendo a 0 sin que las iteraciones se acerquen a ningún punto en particular.

Otra propiedad notable de la suavidad, incluso sin convexidad, es que impide el *overshoot*: con paso $\gamma = 1/L$, el algoritmo no puede sobrepasar un punto crítico. Formalmente:

Lema 3.2. Sea $f: \mathbb{R}^d \rightarrow \mathbb{R}$ diferenciable y $x \in \mathbb{R}^d$ con $\nabla f(x) \neq 0$. Supongamos que f es suave con parámetro L sobre el segmento que conecta x y $x' = x - \gamma \nabla f(x)$, donde $\gamma = 1/L' < 1/L$. Entonces x' tampoco es punto crítico.

En conclusión, bajo suavidad, el descenso por gradiente proporciona tasas de convergencia al gradiente nulo comparables a las del caso convexo, aunque sin garantías de alcanzar el mínimo global.

3.2. Análisis de la trayectoria

Aunque la sección anterior muestra las limitaciones teóricas del descenso por gradiente en el caso no convexo general, en ciertos problemas concretos es posible demostrar que el algoritmo converge al mínimo global, aprovechando propiedades específicas de la función objetivo. Esto se denomina **análisis de la trayectoria**.

Nuestro objetivo va a ser que, partiendo de una región adecuada, las iteraciones del descenso por gradiente permanezcan en una zona donde la función es localmente suave, de forma que así se intenta permanecer en una región favorable que evita ciertos puntos críticos no óptimos. Esto permite establecer convergencia global a pesar de la no convexidad.

Esto es realmente lo que ocurre en el caso de las redes neuronales, por lo que motivaremos nuestro estudio con dicho caso.

3.2.1. Redes neuronales lineales profundas

Consideramos el problema de ajustar una transformación lineal entre entradas $x_i \in \mathbb{R}^d$ y salidas $y_i \in \mathbb{R}^m$. Sean $X \in \mathbb{R}^{d \times n}$ e $Y \in \mathbb{R}^{m \times n}$ las matrices de datos, cuyas columnas corresponden a las muestras de entrada y salida, respectivamente. Este es un problema convexo con solución explícita, equivalente a entrenar una red neuronal lineal de una sola capa [14].

Si en cambio consideramos una red de ℓ capas con matrices de pesos $W_1 \dots W_\ell$, el problema de entrenamiento se convierte en minimizar

$$\|W_\ell W_{\ell-1} \dots W_1 X - Y\|_F^2,$$

que ya no es convexo respecto a los pesos de forma conjunta.

Aunque realmente esto no es una red neuronal como tal, es simplemente una transformación lineal de las $W_\ell \dots W_1$, este ejemplo proporciona un modelo sencillo para estudiar los efectos de la no convexidad en el entrenamiento con descenso por gradiente.

La función que vamos a ver a continuación captura exactamente esta estructura en el caso más simple posible: se va a tratar de una red de d capas con una sola neurona por capa y un único dato de entrenamiento $(1, 1)$.

Como veremos, lo que hace este modelo especialmente útil es que, a pesar de su aparente sencillez, hereda los dos rasgos problemáticos del caso general: la no convexidad (con puntos de silla que no son mínimos globales) y la dependencia crítica de la inicialización. Estudiar cómo se comporta el descenso por gradiente sobre esta función nos permitirá extraer conclusiones relevantes sobre el entrenamiento de redes más complejas, con la ventaja de poder hacerlo de forma totalmente rigurosa.

3.2.2. Una función no convexa modelo

Consideremos la función $f: \mathbb{R}^d \rightarrow \mathbb{R}$ dada por

$$f(x) = \frac{1}{2} \left(\prod_{k=1}^d x_k - 1 \right)^2.$$

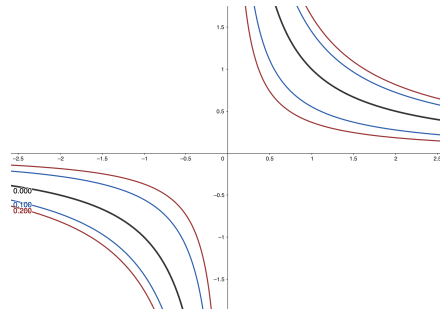


Figura 3.2.: Curvas de nivel de la función f para $d = 2$ (imagen de generación propia usando GeoGebra).

Aquí, cada coordenada x_k juega el papel de la matriz de pesos de la k -ésima capa, y el producto $\prod_k x_k$ representa la transformación global que la red aplica al dato. Minimizar f equivale entonces a encontrar unos pesos cuyo producto sea 1, que es precisamente la solución óptima para ese dato.

Su gradiente es

$$\nabla f(x) = \left(\prod_k x_k - 1 \right) \left(\prod_{k \neq 1} x_k, \dots, \prod_{k \neq d} x_k \right)^\top.$$

Los puntos críticos de f se clasifican según el comportamiento del gradiente:

1. **Mínimos globales:** El conjunto $\{\prod_k x_k = 1\}$, donde $f(x) = 0$.
2. **Puntos de silla:** Aquellos donde el gradiente se anula pero no se alcanza un mínimo. Analíticamente, observando $\nabla f(x)$, el gradiente se anula si al menos dos de las

3. Descenso por gradiente en funciones no convexas

coordenadas de x son iguales a cero, resultando en un valor de la función de $1/2$.

En particular, f no es convexa, por lo que admite puntos críticos que no son mínimos globales.

Nuestro objetivo es demostrar que el descenso por gradiente, partiendo de $x_0 > 0$ con $\prod_k (x_0)_k < 1$, converge al mínimo global.

3.2.3. Suavidad a lo largo de la trayectoria

Supongamos que la función f no es globalmente suave. Vamos a demostrar que lo es a lo largo de las trayectorias del descenso por gradiente que arrancan en la región $X = \{x > 0, \prod_k x_k \leq 1\}$.

El ingrediente clave para que eso funcione es el **c-balanceo**.

Definición 3.3. Sea $x > 0$ (componente a componente) y $c \geq 1$. Diremos que x está **c-balanceado** si

$$x_i \leq cx_j, \quad \forall 1 \leq i, j \leq d.$$

Intuitivamente, el c -balanceo garantiza que las coordenadas de x no difieren demasiado en magnitud, lo que en el contexto de las redes neuronales equivale a que los pesos de las distintas capas permanezcan proporcionales entre sí. En concreto:

Lema 3.4. Sea $x > 0$ c -balanceado con $\prod_k x_k \leq 1$. Entonces, para cualquier tamaño de paso $\gamma > 0$, el punto $x' = x - \gamma \nabla f(x)$ satisface $x' \geq x$ (componente a componente) y es también c -balanceado.

Demostración. Definamos $\Delta := -\gamma(\prod_k x_k - 1)(\prod_k x_k) \geq 0$. La actualización del descenso toma la forma $x'_k = x_k + \Delta/x_k \geq x_k$. Para cada i, j , usando $x_i \leq cx_j$ y $1/x_i \leq c/x_j$, obtenemos

$$x'_i = x_i + \frac{\Delta}{x_i} \leq cx_j + \frac{c\Delta}{x_j} = cx'_j.$$

Por tanto, x' también es c -balanceado. □

A partir de este lema, puede acotarse la norma espectral de la Hessiana sobre la trayectoria: si $x > 0$ es c -balanceado con $\prod_k x_k \leq 1$, entonces $\|\nabla^2 f(x)\| \leq 3dc^2$.

En consecuencia, f es suave con parámetro $L = 3dc^2$ sobre cada segmento $[x_t, x_{t+1}]$ de la trayectoria, lo que justifica aplicar el lema de descenso suficiente en cada paso y elegir el tamaño de paso $\gamma = 1/(3dc^2)$.

3.2.4. Convergencia

Teorema 3.5. Sea $c \geq 1$ y $\delta > 0$ tales que $x_0 > 0$ es c -balanceado con $\delta \leq \prod_k (x_0)_k < 1$. Tomando el tamaño de paso $\gamma = \frac{1}{3dc^2}$, el descenso por gradiente satisface

$$f(x_T) \leq \left(1 - \frac{\delta^2}{3c^4}\right)^T f(x_0), \quad T \geq 0.$$

Demostración. Para cada $t \geq 0$, f es suave sobre $\text{conv}(\{x_t, x_{t+1}\})$ con parámetro $L = 3dc^2$, luego el descenso suficiente da

$$f(x_{t+1}) \leq f(x_t) - \frac{1}{6dc^2} \|\nabla f(x_t)\|^2.$$

Calculando explícitamente el gradiente y usando el c -balanceo con $\delta \leq \prod_k x_k \leq 1$:

$$\|\nabla f(x)\|^2 = 2f(x) \sum_{i=1}^d \left(\prod_{k \neq i} x_k \right)^2 \geq 2f(x) \frac{d}{c^2} \left(\prod_k x_k \right)^2 \geq 2f(x) \frac{d\delta^2}{c^2}.$$

Sustituyendo:

$$f(x_{t+1}) \leq f(x_t) - \frac{1}{6dc^2} \cdot 2f(x_t) \frac{d\delta^2}{c^2} = f(x_t) \left(1 - \frac{\delta^2}{3c^4} \right).$$

Iterando el argumento se obtiene la cota enunciada. $\square$

El resultado muestra que $f(x_T) \rightarrow 0$, es decir, el algoritmo converge al mínimo global con una disminución exponencial del error, comparable a la del caso fuertemente convexo. Sin embargo, la tasa depende de δ , que mide cuán lejos de 0 está el producto $\prod_k x_k$ en el punto inicial.

3.2.5. Dependencia de la inicialización

La calidad del resultado anterior está caracterizada por la elección del punto inicial. Para $x_0 = (1/2, \dots, 1/2)$, tenemos $c = 1$ y $\delta = 1/2^d$, luego el factor de contracción es $(1 - 1/(3 \cdot 4^d))$, y se necesitan aproximadamente $T \approx 4^d$ iteraciones para reducir el error inicial por un factor constante. Este coste exponencial en d hace que el método sea inviable en alta dimensión para inicializaciones genéricas.

Para obtener convergencia en tiempo polinomial, se requiere una δ que decaiga a lo sumo polinomialmente con d . Por ejemplo, tomando $x_i = 1 - b/d$ para $b = \mathcal{O}(1)$ se tiene $\delta \approx e^{-b} = \Omega(1)$, pero el punto inicial se encuentra a distancia $\mathcal{O}(1/\sqrt{d})$ de la solución óptima $(1, \dots, 1)$, es decir, se comienza *cerca* del óptimo.

3.3. Análisis final

Los resultados de este capítulo revelan un panorama más matizado que el del caso convexo. Bajo suavidad, el descenso por gradiente asegura la convergencia de $\|\nabla f(x_t)\|^2$ a 0 a una tasa de $\mathcal{O}(1/T)$, pero sin garantizarnos que alcancemos el mínimo global.

Cuando tenemos alguna otra hipótesis adicional, como en el caso del c -balanceo, es posible demostrar convergencia global con tasas exponenciales, aunque esto nos puede dar problemas si la inicialización es genérica.

Todo esto pone de manifiesto que en la optimización de funciones no convexas, la elección del punto inicial y la comprensión de la geometría de la función explican la importancia de la inicialización cuidadosa de pesos, la normalización de capas y el uso de buenos métodos que buscan garantizar que el descenso por gradiente se comporte bien en el entrenamiento de redes neuronales profundas.

Así, mientras que en el Capítulo 2 el éxito dependía exclusivamente de la geometría global de f , en el escenario no convexo el éxito es una combinación de la geometría local, la inicialización y la dinámica propia de la trayectoria.

4. Conclusiones teóricas

A lo largo de los tres capítulos que construyen la primera parte de este trabajo se ha construido, de forma progresiva y rigurosa, un marco matemático para el análisis del descenso por gradiente en problemas de optimización y, en particular, en el contexto del aprendizaje automático.

A modo de síntesis, se presentan las siguientes conclusiones clave derivadas del análisis teórico:

1. La geometría de la función dicta la velocidad del algoritmo.

El mensaje central de esta parte puede resumirse en una sola frase: **la velocidad de convergencia depende de forma esencial de la geometría de la clase de funciones sobre la que actúa el algoritmo**. Es decir, el éxito de la optimización depende intrínsecamente de la curvatura del espacio.

- **Convexidad pura:** Garantiza que cualquier mínimo local es global, transformando el entrenamiento en la búsqueda de las raíces del gradiente. Sin embargo, sin hipótesis adicionales, los métodos de primer orden se ven limitados a una tasa sublineal de convergencia, equivalente a una complejidad de $\mathcal{O}(1/\varepsilon^2)$ iteraciones para alcanzar un error ε .
- **Suavidad (parámetro L) :** Al acotar superiormente el crecimiento de la función mediante un paraboloide, se formaliza el *descenso suficiente*. Lo que permite que, bajo un tamaño de paso constante ($\gamma = 1/L$), se acelere la convergencia a una tasa sublineal rápida de $\mathcal{O}(1/\varepsilon)$.
- **Convexidad fuerte (parámetro μ):** Al garantizar una curvatura mínima residual, se eliminan regiones demasiado planas. Esto desbloquea la convergencia lineal (exponencial en las iteraciones/lineal en el error) a una tasa de $\mathcal{O}(\log(1/\varepsilon))$, donde la velocidad queda determinada por el número de condición $\kappa = L/\mu$.
- **No diferenciabilidad:** La falta de diferenciabilidad obliga a reemplazar el gradiente por el subgradiente. Aunque esta generalización preserva las cotas de convergencia, impone dos restricciones importantes: el descenso deja de ser monótono en cada iteración, y es necesario trabajar con el promedio de las iteraciones en vez de solo con la última para obtener garantías teóricas.
- **Estocasticidad:** La estocasticidad introduce un ruido (*noise floor*) debido a la varianza que impide la convergencia exacta con un paso constante. La solución teórica consiste en hacer decrecer el tamaño de paso ($\gamma_t \rightarrow 0$), a costa de ralentizar etapas iniciales. Debido a esto, la teoría justifica el uso de *mini-batches* como una herramienta matemática para reducir dicha varianza proporcionalmente al tamaño del bloque m .

La conclusión es clara: más que buscar un algoritmo más sofisticado, la estrategia más eficaz para acelerar el entrenamiento del modelo consiste en garantizar que la función de pérdida pertenezca a una clase geoméricamente favorable.

4. Conclusiones teóricas

2. Los límites del caso no convexo.

Como hemos visto en el Capítulo 3, la ausencia de convexidad no invalida el análisis, pero sí cambia su naturaleza. Por ejemplo, en el caso de funciones suaves no convexas no podemos establecer una cota sobre el error, solo podemos acotar la norma del gradiente (Teorema 3.1), lo que provoca que no podamos garantizar convergencia a un mínimo global: el algoritmo puede detenerse en un punto de silla o en un mínimo local.

Sin embargo, cuando la función posee una estructura adicional (como el c -balanceo) es posible demostrar la convergencia con tasas exponenciales mediante el análisis de la trayectoria, resultado que pone de manifiesto que las propiedades globales de la función pueden recuperarse a través de propiedades locales de la trayectoria cuando se parte desde una buena región de inicialización.

Esta dependencia de la inicialización es uno de los fenómenos más relevantes del caso no convexo: el mismo algoritmo con el mismo tamaño de paso puede converger a un mínimo global o quedar atrapado en un punto de silla según el punto de partida x_0 . Este fenómeno explica la importancia de las técnicas de inicialización de pesos en el entrenamiento de redes neuronales profundas.

3. Conexión con el aprendizaje automático.

Todo el desarrollo de esta parte viene motivado por el problema concreto de minimizar una función de pérdida sobre los parámetros de un modelo. Desde esta perspectiva, los resultados obtenidos responden principalmente a las preguntas del aprendizaje automático:

- *¿Por qué el descenso por gradiente funciona bien en alta dimensión?* Porque su tasa de convergencia es independiente de la dimensión d del espacio de parámetros, como se deduce del Teorema 2.2.
- *¿Por qué la regularización acelera el entrenamiento?* Porque introduce convexidad fuerte, transformando una tasa sublineal en exponencial.
- *¿Por qué se usan mini-batches en la práctica?* Porque combinan la eficiencia por iteración del SGD con la reducción de la varianza propia del gradiente completo.
- *¿Por qué la inicialización importa tanto en redes neuronales?* Porque en el caso no convexo no existe garantía global de convergencia: la región desde la que se parte determina hacia qué punto converge el algoritmo.

En definitiva, los fundamentos matemáticos desarrollados hasta ahora constituyen el lenguaje preciso que permite entender y mejorar el comportamiento de los algoritmos de optimización sobre los que se sustenta el aprendizaje automático moderno.

La segunda parte del trabajo se encargará de validar empíricamente cada uno de los resultados presentados.

Parte II.

Experimentación práctica

5. Experimentación en el caso convexo

En la primera parte del trabajo hemos establecido mediante demostraciones rigurosas las tasas de convergencia del descenso por gradiente bajo distintas hipótesis sobre la función objetivo.

El objetivo de este capítulo es implementar los algoritmos en distintos problemas (primero en funciones muy sencillas y más tarde en datasets reales) y verlos en acción, verificando que el comportamiento empírico se ajusta a las cotas demostradas.

Los experimentos están escritos en Python, más concretamente usando PyTorch [11] con NumPy [7], Matplotlib [8] y scikit-learn [12] que contienen los algoritmos estudiados (descenso por gradiente, Nesterov, subgradiente, SGD y *mini-batch* SGD) y son muy útiles para cálculo, visualización y carga de datos, de modo que cada experimento se reduce a especificar la función, sus derivadas y los parámetros del problema.

Observación 5.1. Al igual que en el Capítulo 2, usaremos $\mathcal{O}(1/\epsilon)$ para referirnos a la complejidad de iteraciones para alcanzar una precisión ϵ y $\mathcal{O}(1/T)$ para referirnos a la tasa de convergencia (como disminuye el error a medida que aumentan las iteraciones).

El código utilizado para obtener todas las Figuras que analizaremos en este capítulo se puede encontrar en el siguiente repositorio: <https://github.com/lidianievasugr/TrabajoFinGrado>

5.1. Validación empírica de las tasas de convergencia

5.1.1. Caso convexo Lipschitz: $\mathcal{O}(1/\sqrt{T})$

Al igual que en el estudio teórico, comenzaremos por el escenario más débil que admite garantías de convergencia: en el que la función es simplemente convexa, diferenciable, y con gradiente acotado por una constante B . Recordemos del Teorema 2.2 que, eligiendo un paso óptimo $\gamma = \frac{R}{B\sqrt{T}}$ (con $R \geq \|x_0 - x^*\|$), se tiene

$$\frac{1}{T} \sum_{t=0}^{T-1} (f(x_t) - f(x^*)) \leq \frac{RB}{\sqrt{T}}.$$

Como función modelo tomamos $f(x) = \sqrt{x_1^2 + x_2^2 + 1} = \sqrt{\|x\|^2 + 1}$ sobre $\mathbb{R}^2$. Es convexa, diferenciable, y su gradiente $\nabla f(x) = x / \sqrt{1 + \|x\|^2}$ verifica $\|\nabla f(x)\| < 1$ para todo x , por lo que podemos tomar $B = 1$.

La Figura 5.1 usa una escala logarítmica en ambos ejes, de manera que la cota $RB/\sqrt{T}$ se manifiesta como una recta de pendiente $-1/2$. La curva empírica reproduce esa misma pendiente, situándose por debajo de la cota teórica tal y como cabe esperar. El experimento confirma así el comportamiento $\mathcal{O}(1/\sqrt{T})$ predicho por el Teorema 2.2. Esta convergencia sin embargo, es muy lenta: para reducir el error a la décima parte se requieren cien veces más iteraciones.

5. Experimentación en el caso convexo

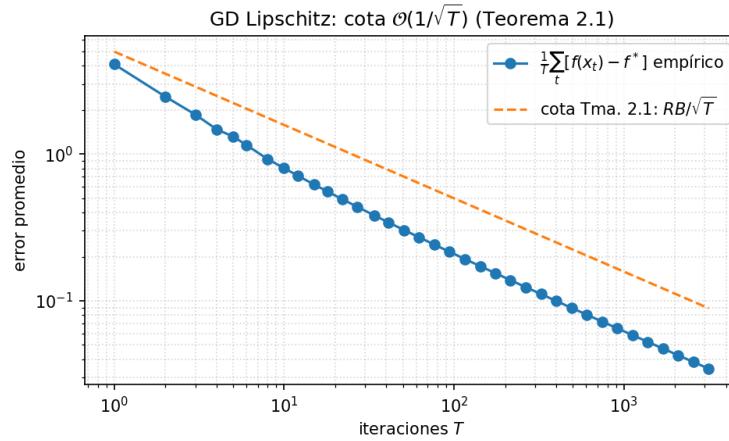


Figura 5.1.: Promedio del error frente a la cota $RB/\sqrt{T}$ para $f(x) = \sqrt{\|x\|^2 + 1}$.

La siguiente sección muestra que añadir una hipótesis de suavidad permite mejorar sensiblemente la tasa.

5.1.2. Caso suave convexo: tasa $\mathcal{O}(1/T)$

Partimos de la cuadrática más sencilla, $f(x) = \frac{1}{2}x^\top Ax$ con A simétrica definida positiva, de forma que se cumplen las hipótesis del Teorema 2.6 (función convexa, suave y diferenciable) con paso $\gamma = \frac{1}{L}$ (donde L es el parámetro de suavidad) de forma que esperamos

$$f(x_T) - f(x^*) \leq \frac{L\|x_0 - x^*\|^2}{2T}, \quad T > 0.$$

La Figura 5.2 ilustra la convergencia del algoritmo de descenso por gradiente (GD) para una función convexa y suave (Definición 2.3), comparando el error empírico con la cota teórica establecida en el Teorema 2.6.

La gráfica emplea una escala logarítmica en ambos ejes, de forma que una relación de potencia $y = x^k$ se manifiesta como una línea recta, donde una pendiente de -1 identifica una caída del error proporcional a $\mathcal{O}(1/T)$.

Como se observa, la **línea naranja** representa el límite superior garantizado por la teoría, dado por la expresión $L\|x_0 - x^*\|^2/(2T)$. Su trayectoria lineal y constante en la gráfica confirma visualmente que el algoritmo posee una convergencia de orden $\mathcal{O}(1/T)$. Por su parte, la **línea azul** muestra el error empírico ($f(x_T) - f(x^*)$), el cual se mantiene consistentemente por debajo de la cota, validando así la robustez del resultado teórico.

Es importante notar que esta cota es *universal*, ya que no requiere hipótesis adicionales sobre la geometría de f más allá de suavidad y convexidad. Sin embargo, el hecho de que el error real sea significativamente menor a la cota sugiere que, cuando existe una estructura más favorable (como una fuerte convexidad $\mu > 0$), el algoritmo experimenta una aceleración que la cota $\mathcal{O}(1/T)$ no logra capturar plenamente, hecho que motiva el estudio del siguiente resultado.

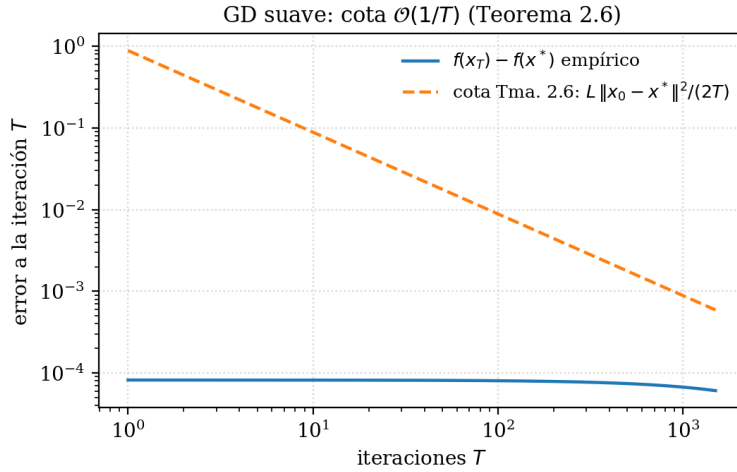


Figura 5.2.: Rapidez con la que el GD reduce el error al buscar el mínimo de una función suave.

5.1.3. Caso suave y fuertemente convexo: tasa $\mathcal{O}((1 - \frac{\mu}{L})^T)$

Bajo la hipótesis adicional de fuerte convexidad (Definición 2.7), el Teorema 2.10 garantiza una convergencia geométrica del error:

$$f(x_T) - f(x^*) \leq \frac{L}{2} \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2, \quad T > 0,$$

donde la tasa de decaimiento exponencial depende directamente del número de condición $\kappa = L/\mu$. Para verificar este comportamiento, se han simulado cuadráticas en $\mathbb{R}^2$ con un parámetro de suavidad $L = 1$ fijo y variando la constante de fuerte convexidad $\mu \in \{0.1, 0.05, 0.01, 0.001\}$, lo que resulta en valores de $\kappa \in \{10, 20, 100, 1000\}$.

La Figura 5.3 ilustra el comportamiento predicho por la teoría. En una escala semilogarítmica (eje y logarítmico), las curvas de error se presentan como líneas rectas, lo que confirma un decaimiento exponencial cuya pendiente está determinada por $\log(1 - 1/\kappa)$.

En el gráfico, las líneas continuas representan el error empírico, mientras que las líneas discontinuas corresponden a las cotas teóricas superiores.

Se observa claramente que el rendimiento del algoritmo es altamente sensible a κ :

- Cuando κ es pequeño (fuerte convexidad dominante), el término $(1 - \mu/L)$ es significativamente menor que 1, lo que provoca que el error se reduzca casi instantáneamente.
- Por el contrario, cuando κ es elevado ($\mu/L \approx 0$), la tasa de convergencia se aproxima a 1, resultando en un descenso mucho más lento del error.

Este experimento pone de manifiesto que la fuerte convexidad permite obtener una convergencia lineal, mucho más rápida que la tasa sublineal $\mathcal{O}(1/T)$ del caso simplemente convexo y suave. No obstante, cuando μ es muy pequeño, es decir, cuando κ es grande, la ventaja práctica de la convergencia lineal puede verse reducida, ya que el factor de contracción se aproxima a 1.

5. Experimentación en el caso convexo

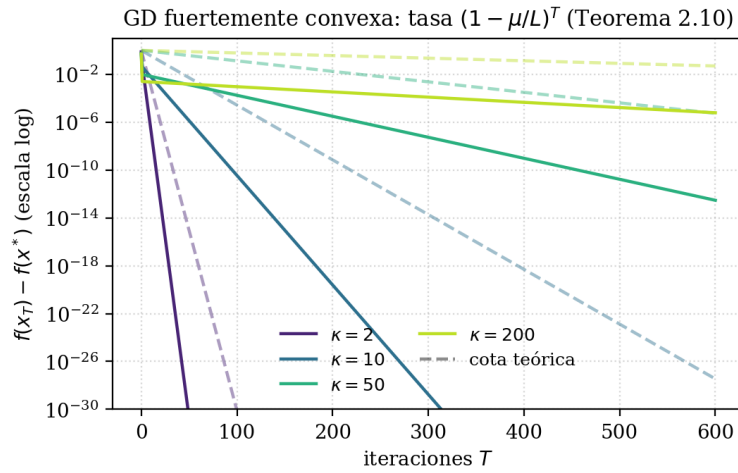


Figura 5.3.: Convergencia exponencial del descenso por gradiente bajo fuerte convexidad para distintos valores de κ .

5.1.4. Aceleración de Nesterov: tasa $\mathcal{O}(1/T^2)$

El Teorema 2.11, bajo las mismas hipótesis anteriores, mejora la tasa hasta $\mathcal{O}(1/T^2)$.

Sobre la misma cuadrática, la diferencia es visible:

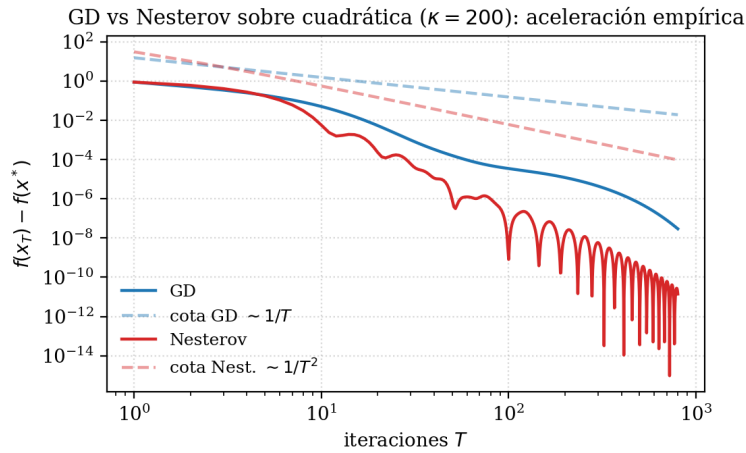


Figura 5.4.: GD frente al método Nesterov sobre una cuadrática con $\kappa = 200$.

La Figura 5.4 muestra claramente la diferencia: las curvas de Nesterov son visiblemente más empinadas, lo cual es coherente con la cota teórica de orden $\mathcal{O}(1/T^2)$. Mientras que el GD reduce el error de forma más gradual, la trayectoria de Nesterov muestra un descenso más pronunciado, coherente con su cota teórica superior. Es interesante notar el comportamiento no monótono de Nesterov (las oscilaciones en la curva roja), un fenómeno típico de los métodos con «momentum» que, aunque sacrifican el descenso estricto en cada iteración,

logran una ganancia global en eficiencia. En este experimento, alcanzar un error de 10^{-4} requiere 30 iteraciones de GD frente a las 18 que necesita Nesterov.

5.1.5. Descenso de subgradiente sobre funciones no diferenciables

Tal y como analizamos en la parte teórica, cuando hablamos de funciones no diferenciables tenemos que pasar a usar el descenso por subgradiente en vez del GD clásico, reemplazando $\nabla f(x)$ por un subgradiente arbitrario $g \in \partial f(x)$.

Como función modelo no diferenciable usaremos $f(x) = |x|$. Para este tipo de funciones, la teoría (Teorema 2.17) establece que el método del subgradiente con un tamaño de paso decreciente garantiza convergencia, alcanzando para el promedio de las iteraciones $(\bar{x}_T = \frac{1}{T} \sum_{t=1}^T x_t)$ un ritmo de $\mathcal{O}(1/\sqrt{T})$.

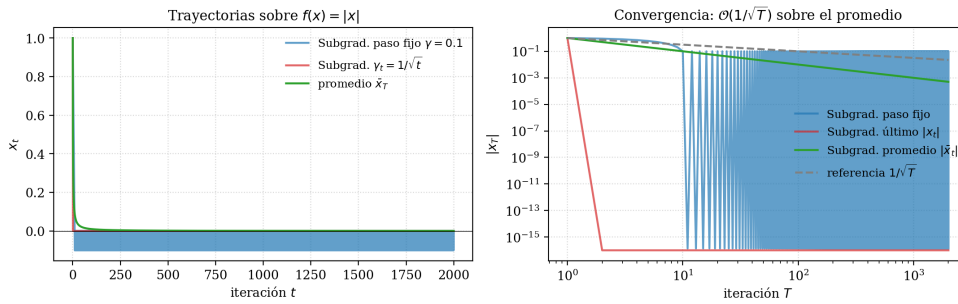


Figura 5.5.: Comparativa entre paso constante (no convergente) y paso decreciente con promediado (convergencia $\mathcal{O}(1/\sqrt{T})$) para $f(x) = |x|$.

La Figura 5.5 permite contrastar estas garantías teóricas con la práctica. En el panel izquierdo se observa que el uso de un paso constante $\gamma = 0.1$ en el método de subgradiente no conduce a una convergencia exacta al óptimo, sino que provoca oscilaciones indefinidas en torno al origen, lo que confirma la necesidad de tratar la no diferenciable de manera específica.

Por el contrario, el panel derecho muestra que el subgradiente con tamaño de paso decreciente, $\gamma_t = 1/\sqrt{t+1}$, sí converge (línea roja), y su promedio (línea verde) lo hace siguiendo la pendiente $1/\sqrt{T}$ predicha por el Teorema 2.17.

Aunque el último iterado puede presentar oscilaciones residuales, el promedio suaviza dichas oscilaciones y permite obtener garantías teóricas de convergencia en este contexto.

5.1.6. Descenso estocástico y *mini-batches*

En aprendizaje automático, la función de pérdida se define típicamente como un promedio de pérdidas individuales: $f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$. Dado que calcular ∇f es muy costoso cuando trabajamos con un gran volumen de datos, el algoritmo de Gradiente Estocástico (SGD) aproxima este valor mediante $\nabla f_{i_t}(x)$ con un índice i_t aleatorio. Los métodos de *mini-batch* refinan esta aproximación promediando un subconjunto B_t de tamaño m . Según los Teoremas 2.20 y 2.22, la varianza de este estimador decae como σ^2/m , un factor crítico que determina el error residual del método.

5. Experimentación en el caso convexo

Para ilustrar este comportamiento, la Figura 5.6 presenta un experimento sobre un problema sintético de mínimos cuadrados ($n = 500, d = 20$) comparando cuatro tamaños de *batch*: $m \in \{1, 16, 64, 500\}$.

Al analizar el coste en términos de evaluaciones de gradiente individual, se observa el contraste fundamental predicho por la teoría: el SGD desciende con mayor rapidez en las etapas iniciales gracias a su bajo coste por iteración, pero las iteraciones acaban estabilizándose alrededor de un *noise floor* debido a la varianza del gradiente estocástico. Aumentar el tamaño del *batch* m reduce esta varianza y desplaza el *noise floor* hacia valores menores, aunque a costa de aumentar el coste computacional de cada iteración. Finalmente, se observa que el Batch GD ($m = 500$) requiere un mayor número de evaluaciones para progresar inicialmente, pero es el único capaz de converger a una precisión arbitraria al no estar limitado por la varianza del muestreo.

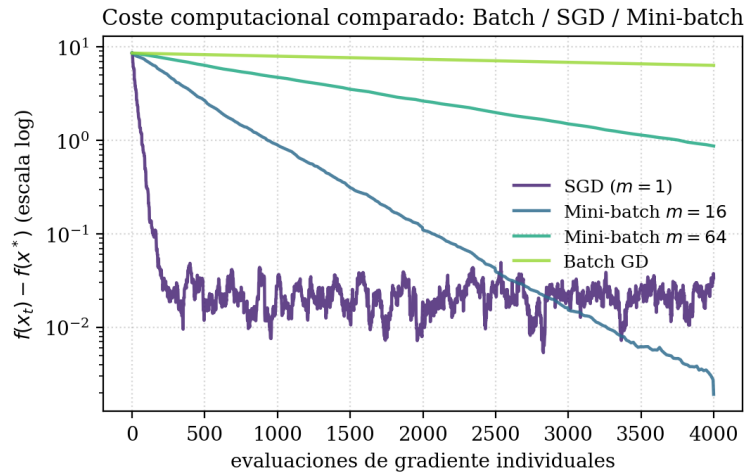


Figura 5.6.: Comparativa de coste computacional para distintos tamaños de *batch* m .

5.2. Aplicaciones al aprendizaje automático

La sección anterior validaba las tasas de convergencia sobre ejemplos muy generales. Aquí damos el siguiente paso: aplicamos los mismos métodos a dos problemas clásicos del aprendizaje automático sobre datos reales y comprobaremos que se comportan como cabía esperar.

5.2.1. Regresión lineal: caso convexo

La regresión lineal por mínimos cuadrados busca un vector de pesos $w \in \mathbb{R}^d$ que minimice

$$\hat{L}(w) = \frac{1}{2n} \sum_{i=1}^n (x_i^\top w - y_i)^2, \quad (5.1)$$

donde $\{(x_i, y_i)\}_{i=1}^n$ es el conjunto de entrenamiento. La ecuación (5.1) es convexa, por lo que podemos calcular el óptimo $\hat{L}^*$ y por tanto medir el error $\hat{L}(w_t) - \hat{L}^*$ con precisión arbitraria.

Como conjunto de datos vamos a utilizar el dataset *Diabetes* [5] (442 muestras, 10 *features*). Es un dataset pequeño, sin valores extremos, en el que cada dato individual es representativo del promedio general. Como la variabilidad entre las muestras es mínima, podemos usar el mismo tamaño de paso para comparar Batch GD y SGD.

En la siguiente Figura vemos sobre este dataset el comportamiento ya visto en la sección anterior:

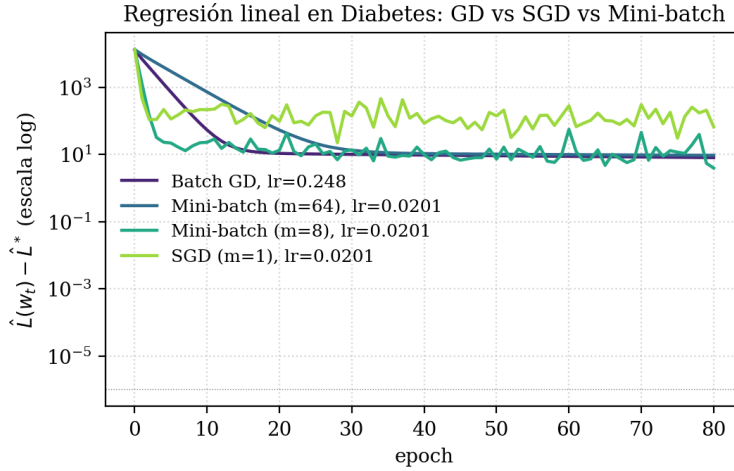


Figura 5.7.: Pérdida $\hat{L}(w_t) - \hat{L}^*$ en escala logarítmica para Batch GD y tres tamaños de *mini-batch* sobre el dataset *Diabetes*.

De nuevo, podemos ver que el Batch GD desciende de forma más suave y estable, al no estar afectado por la varianza del muestreo; los métodos estocásticos (*mini-batch*) descienden más rápido al inicio pero quedan atrapados en su correspondiente *noise floor*.

Esto justifica lo que realmente se hace en la práctica: se comienza con SGD y, una vez próximos al óptimo, se reduce el paso o se aumenta el tamaño del *batch* para obtener una convergencia más precisa.

5.2.2. Regresión logística con regularización L_1 : caso no diferenciable

La regresión logística sobre datos de clasificación binaria $\{(x_i, y_i)\}_{i=1}^n$ con $y_i \in \{-1, +1\}$ minimiza la pérdida

$$L(w) = \frac{1}{n} \sum_{i=1}^n \log(1 + \exp(-y_i x_i^\top w)),$$

función suave y convexa respecto a w .

Normalmente, cuando llevamos esta función a la práctica se desea *seleccionar features*, que quiere decir que el modelo decida cuáles de los datos de entrada son útiles y cuáles son ruido, ignorando estos últimos por completo. Cuando queremos hacer esto, añadimos un término de regularización L_1 :

$$L_\lambda(w) = L(w) + \lambda \|w\|_1. \quad (5.2)$$

La función (5.2) sigue siendo convexa pero ya no es diferenciable en los puntos donde

5. Experimentación en el caso convexo

algún $w_j = 0$, es decir, tendremos que hacer uso del subgradiente y promediar los resultados como vimos en el Capítulo 2.

Para ilustrar el método utilizamos el dataset *Breast Cancer Wisconsin* [15] (569 muestras, 30 features), un problema clínico clásico de clasificación binaria. Comparamos tres situaciones: el GD suave sobre L (sin regularización), el último iterado del subgradiente y el promedio del subgradiente con $\lambda = 0.02$.

La Figura 5.8 resume los resultados:

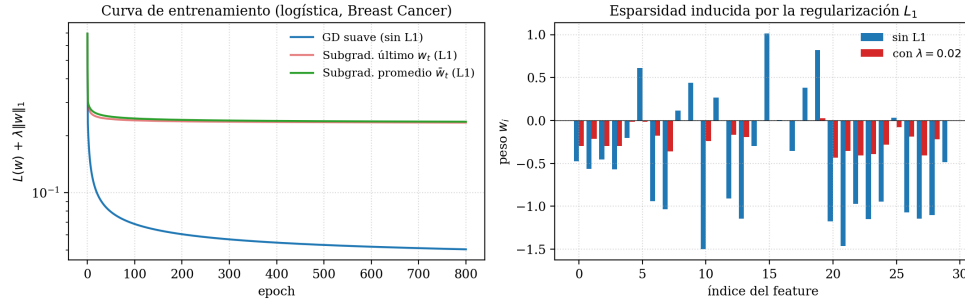


Figura 5.8.: Izquierda: Curvas de entrenamiento sobre *Breast Cancer*. Derecha: Pesos finales del modelo.

En la curva de entrenamiento podemos ver que el subgradiente promediado se sitúa por debajo del último iterado, en concordancia con la teoría.

En el panel derecho, vemos el efecto de la regularización L_1 : del total de *features*, 29 tienen peso no nulo en el modelo sin regularización, frente a 21 en el modelo regularizado. Numericamente la precisión sobre los datos de entrenamiento es del 98.77 % sin regularización y del 97.89 % con $\lambda = 0.02$; una pérdida marginal que se compensa con un modelo notablemente más sencillo y menos propenso al sobreajuste.

Estas observaciones cubren completamente el escenario convexo. En el siguiente capítulo estudiaremos qué queda en pie cuando la función objetivo deja de ser convexa, dando paso a las redes neuronales.

6. Experimentación no convexa

Tras haber validado en el capítulo anterior las garantías teóricas del descenso por gradiente sobre funciones convexas, en este capítulo abordaremos el escenario no convexo. Aquí las cotas dejan de hablar del error $f(x_t) - f(x^*)$ (ya que como vimos en el Capítulo 3 no se garantiza que el algoritmo alcance el mínimo global) y pasan a hablar de la norma del gradiente, salvo en aquellos casos particulares en los que un análisis de la trayectoria nos devuelve una garantía global.

De nuevo, primero contrastaremos las cotas teóricas sobre funciones modelo y, después, aplicaremos el algoritmo a un problema real de aprendizaje automático no convexo: el entrenamiento de una red neuronal.

El código utilizado para obtener todas las figuras que analizaremos en este capítulo se puede encontrar en el siguiente repositorio: <https://github.com/lidianievasugr/TrabajoFinGrado>

6.1. Validación empírica de las cotas no convexas

6.1.1. Función suave no convexa: tasa $\mathcal{O}(1/T)$

En el caso no convexo, comenzamos el estudio por las funciones suaves. El primer resultado que obtuvimos fue el Teorema 3.1, que establece que, para una función diferenciable y suave con parámetro L y un tamaño de paso $\gamma = 1/L$, el descenso por gradiente satisface

$$\frac{1}{T} \sum_{t=0}^{T-1} \|\nabla f(x_t)\|^2 \leq \frac{2L}{T} (f(x_0) - f(x^*)), \quad T > 0.$$

Esto implica que $\|\nabla f(x_t)\|^2 \rightarrow 0$ a una tasa $\mathcal{O}(1/T)$, sin garantizar la convergencia al mínimo global.

Como función modelo vamos a tomar

$$f(x_1, x_2) = \frac{1}{4}((x_1^2 - 1)^2 + (x_2^2 - 1)^2),$$

que cumple todas las hipótesis anteriores. Tiene cuatro mínimos globales en $(\pm 1, \pm 1)$ con $f^* = 0$, un máximo local en el origen y cuatro puntos de silla en $(\pm 1, 0)$ y $(0, \pm 1)$. Sobre la región acotada en la que se mueve la trayectoria se tiene $\|\nabla^2 f(x)\| \leq 5$, así que tomamos $L = 5$ y $\gamma = 1/5$.

En el gráfico izquierdo de la Figura 6.1 podemos ver en escala log-log el promedio empírico de $\|\nabla f(x_t)\|^2$ frente a la cota teórica. Podemos ver que, tras un pequeño tramo inicial en el que el algoritmo se aleja del máximo de origen, la curva empírica adopta una pendiente de -1 , lo que corresponde a una tasa de $\mathcal{O}(1/T)$ y se mantiene siempre por debajo de la cota teórica.

6. Experimentación no convexa

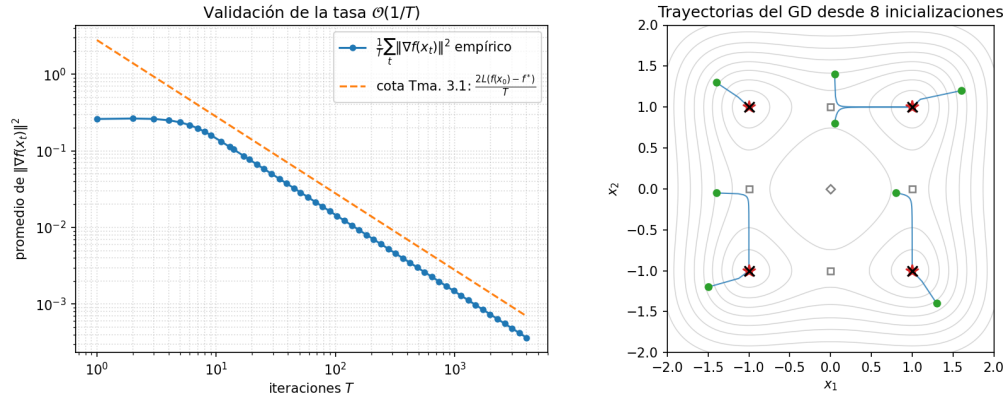


Figura 6.1.: Izquierda: validación de la cota del Teorema 3.1. Derecha: trayectorias del GD desde ocho inicializaciones distintas.

En el panel derecho vemos representadas las trayectorias del GD desde ocho inicializaciones distintas; los círculos verdes marcan el inicio y las estrellas rojas el iterado final, las cruces negras son los mínimos globales y los marcadores grises los puntos de silla. Con esta figura podemos garantizar que el hecho de que la norma del gradiente tienda a cero no equivale a garantizar la convergencia a un mínimo global. Las trayectorias que arrancan en regiones genéricas convergen a uno de los cuatro mínimos, pero las que arrancan cerca de los puntos de silla pueden aproximarse a regiones donde el gradiente es pequeño sin que ello implique optimalidad global.

Esto muestra el principal obstáculo del escenario no convexo: la convergencia del gradiente a 0 basta para detener el algoritmo, pero no para que la solución sea óptima.

6.1.2. Análisis de la trayectoria: tasa exponencial sobre la red lineal profunda

El segundo resultado sobre convergencia que obtenemos en el Capítulo 3 lo obtenemos estudiando la función modelo $f(x) = \frac{1}{2} \left(\prod_{k=1}^d x_k - 1 \right)^2$.

El Teorema 3.5 garantiza, partiendo de un $x_0 > 0$, c -balanceado con $\delta \leq \prod_k (x_0)_k \leq 1$ y su paso $\gamma = 1/(3dc^2)$,

$$f(x_T) \leq \left(1 - \frac{\delta^2}{3c^4} \right)^T f(x_0), \quad T > 0.$$

Para $d=2$ esta función se reduce a $f(x_1, x_2) = \frac{1}{2}(x_1 x_2 - 1)^2$, cuyo conjunto de mínimos globales es la hipérbola $\{x_1 x_2 = 1\}$ del primer cuadrante (más su simétrica). En la Figura 6.2 vemos los resultados del análisis de su convergencia.

En el panel izquierdo vemos que la convergencia es exponencial, en concordancia con la del Teorema. Aunque la curva empírica desciende con una pendiente mucho más pronunciada que la teórica, lo que nos indica que la cota es válida pero conservadora, usando la geometría real de la función obtenemos un resultado mucho más favorable.

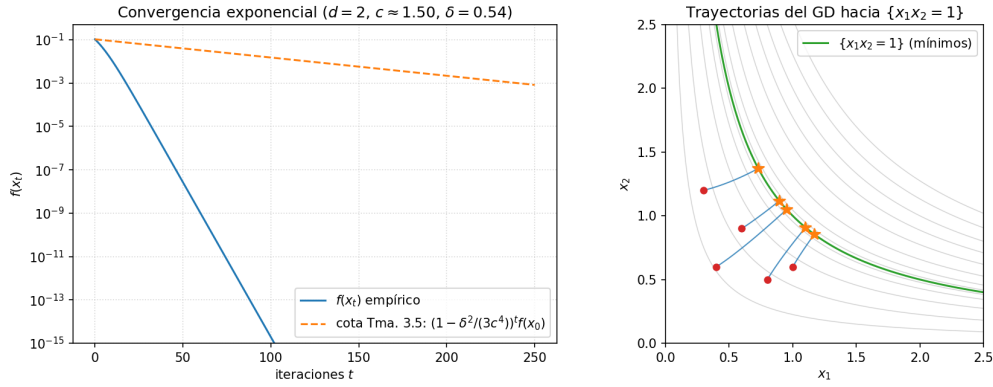


Figura 6.2.: Izquierda: validación de la cota del Teorema 3.5. Derecha: trayectorias del GD desde cinco puntos iniciales distintos.

La Figura de la derecha representa varias trayectorias (líneas azules) sobre las curvas de nivel de f . La curva verde marca la variedad de los mínimos globales y, como predice el Teorema 3.5, todas las trayectorias convergen a dicha variedad. Aunque no todas convergen al mismo punto (el conjunto de mínimos no es un punto aislado), tenemos garantía de optimalidad global pese a la no convexidad.

Tras este resultado damos por finalizado el contraste entre teoría y práctica del caso no convexo. La siguiente sección nos lleva al escenario real en el que se aplican estas ideas: el entrenamiento de redes neuronales.

6.2. Aplicación a redes neuronales

6.2.1. Breve introducción a las redes neuronales

Una **red neuronal artificial** (NN) es un modelo que compone, alternativamente, transformaciones afines y funciones no lineales. En su forma más sencilla conocida como *Multilayer Perceptron* (MLP), una red de ℓ capas se escribe

$$\hat{y}(x; w) = W_\ell \sigma(W_{\ell-1} \sigma(\dots \sigma(W_1 x + b_1) \dots) + b_{\ell-1}) + b_\ell,$$

donde W_i y b_i son las matrices de pesos y los vectores de sesgos de la i -ésima capa, y σ es una función no lineal aplicada componente a componente [1].

Dado un conjunto de entrenamiento $\{(x_i, y_i)\}_{i=1}^n$, el problema de aprendizaje consiste en minimizar la **pérdida empírica**

$$\hat{L}(w) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}(x_i; w), y_i), \quad (6.1)$$

donde ℓ es una función de pérdida (en regresión normalmente $\ell(\hat{y}, y) = (\hat{y} - y)^2$).

Como vimos en el Capítulo 3, la función (6.1) es **no convexa** respecto a w debido a la composición de no lineales σ , y posee varios mínimos locales y puntos de silla.

6. Experimentación no convexa

A pesar de la teoría ya vista, la combinación de descenso por gradiente estocástico con *mini-batches*, una buena inicialización de pesos y un tamaño de paso adecuado conduce, en la práctica, a soluciones que funcionan. La justificación rigurosa de este hecho excede de los objetivos de este trabajo, pero sí podemos verificar empíricamente el Teorema 3.1: aunque no podamos garantizar la convergencia a un mínimo global, sí esperamos que $\|\nabla \hat{L}(w_t)\|^2 \rightarrow 0$ con una tasa de convergencia de $\mathcal{O}(1/T)$.

6.2.2. Implementación para una red neuronal multicapa

Como caso de estudio concreto vamos a implementar una Red Neuronal Perceptrón Multicapa (MLP) a un problema de regresión utilizando la librería PyTorch. Vamos a emplear el conjunto *California Housing* [10] ($n = 20640$ muestras, $d = 8$ atributos) que predice el precio de la vivienda.

Sobre estos datos entrenamos un MLP. La función de pérdida es (6.1) y el optimizador es el descenso por gradiente estocástico con *mini-batches* de tamaño 128. Comparamos tres tamaños de paso $\gamma \in \{10^{-1}, 10^{-2}, 10^{-3}\}$, durante 80 épocas.

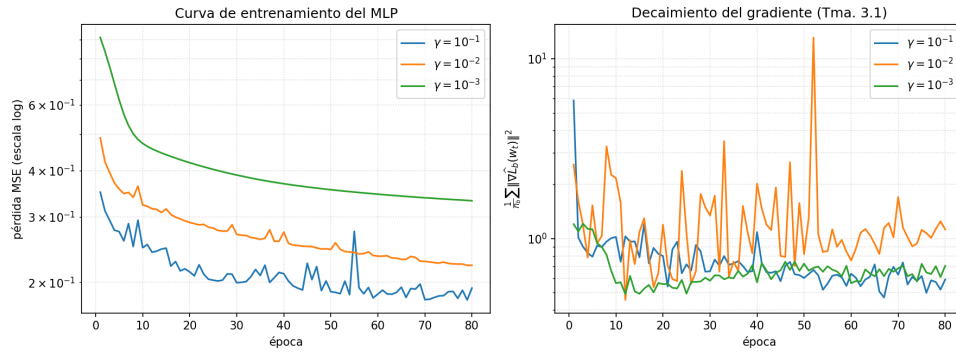


Figura 6.3.: Izquierda: pérdida MSE sobre los datos de entrenamiento. Derecha: promedio por época del cuadrado de la norma del gradiente del *mini-batch*.

La Figura 6.3 muestra los resultados de este estudio.

A la izquierda vemos que las tres configuraciones reducen la pérdida, pero a velocidades muy distintas. Podemos ver que con un tamaño de paso $\gamma = 10^{-1}$ desciende de manera estable y alcanza menor pérdida final, mientras que $\gamma = 10^{-3}$ empieza descendiendo muy lentamente porque la red está demasiado cerca de su inicialización para que el gradiente la saque de esa región plana. En el caso intermedio de $\gamma = 10^{-2}$ obtenemos mayores oscilaciones propias de la varianza del SGD.

En el panel derecho ponemos en práctica el Teorema 3.1: el promedio del cuadrado de la norma del gradiente decae globalmente a lo largo del entrenamiento, en concordancia con la teoría. La curva no es monótona (consecuencia del muestreo aleatorio de *mini-batches*), pero la tendencia es clara y reproduce el patrón de la función modelo ya visto.

Es importante destacar que un gradiente nulo no garantiza haber alcanzado un mínimo global, sino únicamente un punto estacionario, que en redes neuronales suele corresponder a un mínimo local. Además, la variabilidad introducida por el uso de *mini-batches* hace conveniente reducir progresivamente el tamaño de paso o aumentar el tamaño del batch para mejorar la precisión de la solución cerca del óptimo.

7. Conclusiones de la experimentación

A lo largo de los Capítulos 5 y 6 se han validado empíricamente los resultados establecidos en la primera parte del trabajo, primero en el escenario convexo y después en el no convexo. Este último capítulo recoge de forma unificada las conclusiones más relevantes de ambos bloques experimentales:

1. **Las hipótesis sobre la función dictan el orden de convergencia.** En el caso convexo, las curvas empíricas reproducen con precisión las pendientes predichas en escala logarítmica. En el caso no convexo, aunque la media de progreso deja de ser el error $f(x_t) - f(x^*)$ y pasa a ser la norma del gradiente, la tasa del Teorema 3.1 se reproduce fielmente tanto sobre funciones modelo como durante el entrenamiento del MLP sobre *California Housing*. En todos los casos, cuanto más restrictivas son las hipótesis geométricas, mayor es la pendiente observada, en total coherencia con la tabla de tasas óptimas 2.
2. **Las cotas son conservadoras pero universales.** En todos los experimentos realizados el error empírico se sitúa de forma sistemática por debajo de la cota, es decir, las cotas son universales. Esto sugiere que, en la práctica, cuando la función posee estructura adicional favorable (como ocurre en el caso de la red lineal profunda), un descenso por gradiente bien afinado puede rendir mucho mejor de lo que su cota más ajustada predice.
3. **El paso del problema sintético a los datos reales no rompe la teoría.** En los tres datasets empleados, *Diabetes*, *Breast Cancer Wisconsin* y *California Housing*, los algoritmos reproducen exactamente las mismas dinámicas observadas sobre las funciones modelo. La regresión lineal reprodujo el contraste entre Batch GD y SGD con su correspondiente *noise floor*; la regresión logística con regularización L_1 confirmó la necesidad de realizar el promedio en el caso no diferenciable; y el MLP mostró que la norma del gradiente decae a pesar del ruido inducido por los *mini-batches*. Esto indica que las herramientas desarrolladas en la parte teórica son muy útiles para razonar sobre el entrenamiento real de modelos.
4. **La estocasticidad introduce un *noise floor* que la varianza del batch controla.** En todos los experimentos estocásticos se observa un descenso rápido en etapas iniciales seguido de una estabilización alrededor de un error residual cuya magnitud depende directamente del tamaño del batch m . Aumentar m reduce la varianza y desplaza ese suelo hacia valores menores, mientras que el Batch GD es el único capaz de converger con precisión arbitraria (sin *noise floor*). Esto justifica que en la práctica normalmente se comienza con SGD o *mini-batches* pequeños para progresar rápido y, una vez próximos al óptimo, se aumenta el batch o se reduce el tamaño de paso para afinar la solución.
5. **La convergencia de la norma del gradiente a cero no garantiza alcanzar un mínimo global.** Este es el mensaje más relevante del bloque no convexo. Las trayectorias que arrancan cerca de puntos de silla pueden quedar atrapadas en ellos a pesar de que la

7. Conclusiones de la experimentación

norma del gradiente converge a cero. En el entrenamiento del MLP, el tamaño de paso es el factor determinante; un paso muy pequeño impide salir de regiones planas en etapas iniciales, mientras que uno demasiado grande produce oscilaciones que impiden llegar a una solución precisa. En problemas reales esto indica la importancia de elegir un buen tamaño de paso.

En definitiva, los dos capítulos experimentales cierran el ciclo iniciado en la parte teórica: las garantías matemáticas no son solo resultados abstractos, sino predicciones precisas del comportamiento observable de los algoritmos. La teoría se sostiene, las cotas son útiles aunque conservadoras, y los algoritmos muestran en problemas reales exactamente los comportamientos que la geometría de la función anticipaba.

Bibliografía

- [1] C. M. Bishop and H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.
- [2] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.
- [3] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [4] A.-L. Cauchy. Méthode générale pour la résolution des systèmes d'équations simultanées. *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences*, 25:536–538, 1847.
- [5] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004.
- [6] B. Gärtner and M. Jaggi. Optimization for machine learning (lecture notes cs-439, spring 2025), February 2025.
- [7] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, and N. J. Smith. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [8] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [9] Y. Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269(3):543–547, 1983.
- [10] R. K. Pace and R. Barry. Sparse spatial autoregressions. *Statistics and Probability Letters*, 33(3):291–297, 1997.
- [11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. K.öpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, and V. Dubourg. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [13] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [14] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014.
- [15] W. N. Street, W. H. Wolberg, and O. L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. In *IS&T/SPIE 1993 International Symposium on Electronic Imaging: Science and Technology*, volume 1905, pages 861–870, 1993.
- [16] M. Tárraga Navarro. Desigualdades de tipo Brunn-Minkowski vía las medias de números reales. Trabajo fin de máster, Facultad de Matemáticas, Universidad de Murcia, 2017.
- [17] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023.